

CocoSense: AI-Powered Drone-Based System for Comprehensive Coconut Tree Health Monitoring and Yield Prediction

Drone-Based Pest Detection using Deep Learning

Ravindu Pasanjith
(IT22898920)

BSc (Hons) Degree in Information Technology Specializing in
Information Technology

Faculty of Computing
Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

CocoSense: AI-Powered Drone-Based System for Comprehensive Coconut Tree Health Monitoring and Yield Prediction

Drone-Based Pest Detection using Deep Learning

Ravindu Pasanjith
(IT22898920)

Dissertation submitted in partial fulfilment of the requirements for the Degree of
Bachelor of Science (Hons) in Information Technology Specializing in Information
Technology

Faculty of Computing
Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECLARATION

I declare that this is my own work, and this dissertation does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text. Also, I hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute my dissertation in whole or part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as article or books).

Name	Student ID	Signature
Pasanjith W.A.R	IT22898920	

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

Signature of the Supervisor

Date

Signature of the Co-Supervisor

Date

ACKNOWLEDGEMENT

I would first like to express my sincere gratitude to my supervisor, Ms. Uthpala Samarakoon, for her invaluable guidance, continuous encouragement, and unwavering support throughout the duration of this research project. Her expertise in Computer Vision and Artificial Intelligence, together with her constructive feedback, was instrumental in shaping the direction and quality of this work. I am equally grateful to my co-supervisor, Ms. Wishalya Tissera, for her insights, encouragement, and dedication that greatly contributed to the success of this research.

Special thanks are extended to my research team members – Minol Subasinghe, Ranugi Panditharathne, and Tharindu Nadun – for their collaborative spirit, hard work, and mutual support throughout the CocoSense project. The collective effort invested in dataset collection, system integration, and knowledge exchange was indispensable to achieving the goals of this research.

I also thank the plantation owners and agricultural officers from the Coconut Research Institute of Sri Lanka (CRISL) who generously allowed drone image collection in their plantations and provided expert guidance on coconut pest identification. Their domain knowledge was critical to the success of the dataset construction.

I extend my appreciation to the Faculty of Computing at SLIIT for providing access to the computational resources, laboratory facilities, and academic support necessary for conducting this research.

Finally, I am deeply grateful to my parents and family for their unwavering encouragement, patience, and moral support throughout my academic journey.

ABSTRACT

Coconut plantations in Sri Lanka face significant economic losses due to destructive pest infestations, including the coconut mite (*Aceria guerreronis*), the black-headed caterpillar (*Opisina arenosella*), and the spiralling white fly (*Aleurodicus dispersus*), with yield losses reaching 40–80% if undetected. Traditional manual inspection methods are labour-intensive, time-consuming, and impractical at the scale of large plantations, highlighting the urgent need for automated, intelligent pest detection solutions.

This research presents the design, development, and evaluation of a Drone-Based Pest Detection component within the CocoSense system – an AI-powered platform for comprehensive coconut tree health monitoring. A novel dual-model deep learning architecture is proposed: (1) a dedicated MobileNetV2 model with Focal Loss for 3-class coconut mite detection (coconut_mite, healthy, not_coconut), achieving 91.44% test accuracy with a tuned detection threshold of 0.10 and $5\times$ boost factor yielding 79% mite recall; and (2) a MobileNetV2 unified model with Focal Loss for 4-class caterpillar and white fly detection (caterpillar, healthy, not_coconut, white_fly), achieving 96.08% test accuracy and 92.38% macro F1-score. Both models employ transfer learning from ImageNet and Focal Loss ($\gamma=2.0$, $\alpha=0.25$) to overcome the limitations of small, domain-specific datasets.

A key innovation of this research is an intelligent cross-validation mechanism between the two models that reduces mite false positives by 83% and improves overall classification accuracy from 89.7% (best single-model baseline) to 94.8%. To address the absence of geographic context in existing pest detection systems, a custom IoT GPS device was designed and implemented using the ESP32 microcontroller and u-blox NEO-M8N multi-constellation GNSS module, achieving 1.8-metre average positioning accuracy for real-time geospatial mapping of pest-affected trees. A Flask REST API (v9.0) delivers inference results with an average response time of 1.82 seconds, integrated with the CocoSense React Native mobile application for field-deployable use.

A survey conducted among 60 coconut farmers and agricultural officers confirmed strong demand for automated pest detection tools and highlighted the lack of awareness about technology-based solutions among the farming community. A commercialisation plan targeting plantation companies, government agricultural departments, and individual smallholder farmers is proposed through a tiered SaaS model.

Keywords – drone-based pest detection, deep learning, MobileNetV2, transfer learning, Focal Loss, threshold tuning, IoT GPS, ESP32, NEO-M8N, precision agriculture, coconut plantation, Sri Lanka.

1 Table of Contents

2	INTRODUCTION.....	12
2.1	Background.....	12
2.2	Literature Survey.....	14
2.3	Background Survey.....	15
2.4	Research Gap.....	17
2.5	Research Problem	19
3	OBJECTIVES.....	21
3.1	Main Objective	21
3.2	Specific Objectives.....	21
4	SYSTEM METHODOLOGY	22
4.1	System Overview	23
4.2	Pest Detection Component Design and Key Pillars	25
4.2.1	Key Pillar 1: Transfer Learning with Dual-Model Deep Learning.....	26
4.2.2	Key Pillar 2: Intelligent Cross-Validation Between Models.....	29
4.2.3	Key Pillar 3: IoT GPS Tree Location Mapping.....	31
4.3	Requirements.....	32
4.3.1	Functional Requirements.....	32
4.3.2	Use Cases Diagram.....	32
4.3.3	System Use Case Scenarios.....	33
4.3.4	User Requirements	34
4.3.5	Non-Functional Requirements	35
4.3.6	System Requirements.....	35
4.3.7	Challenges.....	36
4.4	Social, Ethical and Security Aspects.....	37
4.4.1	Social Impact.....	37
4.4.2	Ethical Considerations	38
4.4.3	Security Aspects.....	38
4.5	Commercialization Aspects of the Product.....	39
4.5.1	Target Market and Pricing Model.....	39
4.6	Testing and Implementation.....	40

4.6.1	Implementation.....	40
4.6.2	Testing	44
4.6.3	Functional Testing	46
4.6.4	Non-Functional Testing.....	47
4.7	Work Breakdown Structure.....	47
4.8	Gantt Chart	47
5	RESULT AND DISCUSSION	48
5.1	Results.....	48
5.1.1	MobileNetV2 Mite Detection Model v10.....	48
5.1.2	MobileNetV2 Unified Pest Detection Model v1.....	50
5.1.3	Cross-Validation Logic Performance.....	53
5.1.4	IoT GPS Device Performance	54
5.2	Research Findings	55
5.3	Discussion.....	56
5.3.1	Effectiveness of Dual-Model Architecture.....	56
5.3.2	Practical Significance of Cross-Validation.....	56
5.3.3	IoT GPS as a Unique Value Proposition	57
5.3.4	Alignment with Research Objectives	57
6	CONCLUSION	58
6.1	Limitations	59
6.2	Future Work.....	59
	REFERENCES.....	60
	APPENDICES.....	63

LIST OF FIGURES

Figure 1: Coconut Pest Damage – Visual Characteristics of Mite, Caterpillar, and White Fly	13
Figure 2: Background Survey Results – Coconut Farmers (n=40).....	17
Figure 3: Background Survey Results – Agricultural Officers (n=20)	17
Figure 4: Agile (Scrum) + CRISP-DM Methodology Workflow – CocoSense Pest Detection Research	23
Figure 5: CocoSense Pest Detection Component – System Architecture Diagram.....	24
Figure 6: CocoSense Mobile App – Login, Dashboard, and GPS Map Screens	25
Figure 7: MobileNetV2 Mite Model v10 – Architecture and Custom Classification Head	27
Figure 8: MobileNetV2 Unified Model v1 – Architecture and Custom Classification Head	27
Figure 9: Dataset Class Distribution – Unified Model Training Set.....	28
Figure 10: Smart Cross-Validation Logic Flowchart – /predict/all Endpoint.....	30
Figure 11: Custom IoT GPS Device Prototype – ESP32-WROOM-32 + u-blox NEO-M8N GNSS Module.....	31
Figure 12: Use Case Diagram – CocoSense Pest Detection Component.....	33
Figure 13: Mite Detection Threshold Analysis – Recall and Precision vs Threshold Value....	41
Figure 14: IoT GPS Device Firmware Flowchart (ESP32 + NEO-M8N)	42
Figure 15: Flask API v9.0 – /predict/all Endpoint JSON Response Example.....	43
Figure 16: Mobile App – All-Pest Analysis, Treatment Plan, and AI Assistance Screens	44
Figure 17: MobileNetV2 Mite Model v10 – Training and Validation Accuracy/Loss Curves.	49
Figure 18: MobileNetV2 Mite Model v10 – Confusion Matrix (Test Set, n=374)	49
Figure 19: Mobile App Result Screen – Coconut Mite Detected (84.3% Confidence)	50
Figure 20: MobileNetV2 Unified Model v1 – Training and Validation Accuracy/Loss Curves	51
Figure 21: MobileNetV2 Unified Model v1 – Confusion Matrix (Test Set, n=586)	52
Figure 22: MobileNetV2 Unified Model v1 – Per-Class Performance Metrics.....	52
Figure 23: Mobile App Result Screens – Caterpillar White Fly Healthy Not Coconut	53
Figure 24: Mobile App – GPS Plantation Map and Detection Scan History Screens.....	55

LIST OF TABLES

Table 1: Comparison of Existing Pest Detection Systems	18
Table 2: Key Pillars Summary.....	25
Table 3: Dataset Class Distribution	28
Table 4: Use Case Scenario – Attempt Pest Detection Scan.....	33
Table 5: Use Case Scenario – View GPS-Tagged Detection History	34
Table 6: Use Case Scenario – Generate Pest Detection Report	34
Table 7: Non-Functional Requirements	35
Table 8: System Requirements	35
Table 9: Proposed SaaS Pricing Tiers.....	40
Table 10: Pest Detection Test Case – Coconut Mite (Positive)	44
Table 11: Pest Detection Test Case – Coconut Mite (Negative / Healthy)	45
Table 12: Pest Detection Test Case – Caterpillar (Positive).....	45
Table 13: Pest Detection Test Case – White Fly (Positive).....	45
Table 14: Non-Coconut Rejection Test Case.....	46
Table 15: Cross-Validation Logic Test Case.....	46
Table 16: MobileNetV2 Mite Model v10 – Per-Class Performance Metrics (Test Set, n=374)	48
Table 17: MobileNetV2 Unified Model v1 – Per-Class Performance Metrics (Test Set, n=586)	51
Table 18: Cross-Validation Logic – Edge Case Performance (n=100)	53
Table 19: IoT GPS Device Field Performance Summary (n=200 samples).....	54
Table 20: Algorithm Comparison for Coconut Pest Classification	55

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CRISL	Coconut Research Institute of Sri Lanka
CRISP-DM	Cross-Industry Standard Process for Data Mining
DL	Deep Learning
ESP32	Espressif Systems ESP32 Microcontroller
F1	F1-Score (Harmonic Mean of Precision and Recall)
Flask	Python Micro Web Framework
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
GPU	Graphics Processing Unit
HDOP	Horizontal Dilution of Precision
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
IoT	Internet of Things
JSON	JavaScript Object Notation
ML	Machine Learning
MobileNetV2	Mobile Neural Network Version 2 (Sandler et al., 2018)
NEO-M8N	u-blox NEO-M8N Multi-Constellation GNSS Module
NMEA	National Marine Electronics Association
REST	Representational State Transfer
ReLU	Rectified Linear Unit
RGB	Red-Green-Blue colour model
SaaS	Software as a Service
SDLC	Software Development Life Cycle
SLIIT	Sri Lanka Institute of Information Technology
UAV	Unmanned Aerial Vehicle
UART	Universal Asynchronous Receiver-Transmitter
UI	User Interface

LIST OF APPENDICES

Appendix A: Application Logo (CocoSense).....	63
Appendix B: Gantt Chart.....	63
Appendix C: Work Breakdown Structure.....	64
Appendix D: Survey Questionnaire – General Public (Coconut Farmers)	64
Appendix E: Survey Questionnaire – Agricultural Officers.....	66
Appendix F: Dataset Sample Images – Pest Classes.....	69
Appendix G: IoT GPS Device Circuit Diagram.....	70
Appendix H: Flask API v9.0 Endpoint Documentation.....	70
Appendix I: Mobile Application Screenshots.....	73
Appendix J: Plagiarism Report.....	74

2 INTRODUCTION

2.1 Background

In Sri Lanka, coconut (*Cocos nucifera* L.) is one of the most economically important agricultural commodities, ranking as the world's fourth-largest coconut producer with approximately 400,000 hectares under cultivation and producing over 2,700 million nuts annually [1]. The industry directly supports the livelihoods of more than 2.5 million people and contributes significantly to national export revenue. Despite this economic importance, coconut plantations are severely threatened by recurring pest infestations that can cause yield losses of 40–80% if left undetected and untreated [2].

The three primary pest threats are the coconut mite (*Aceria guerreronis* Keifer), the black-headed caterpillar (*Opisina arenosella* Walker), and the spiralling white fly (*Aleurodicus dispersus* Russell). The coconut mite – a microscopic eriophyid mite – feeds on the outer surface of developing coconut nuts, causing characteristic bronzing, surface roughening, and premature nut fall. Because of its microscopic size and subtle early-stage symptoms, mite infestation is extremely difficult to detect by naked-eye inspection, often going unnoticed until widespread plantation damage has already occurred [3]. The black-headed caterpillar feeds on coconut leaf surfaces, causing progressive skeletonisation and chlorosis that ultimately leads to tree death in severe cases. The spiralling white fly infests both coconut and other crops, depositing sooty mould that blocks sunlight from leaves and impairs photosynthesis, weakening tree vigour over time.

Traditional pest management in Sri Lanka relies on periodic visual inspection by trained agricultural officers from the Coconut Research Institute of Sri Lanka (CRISL) and the Department of Agriculture. This approach is inherently time-consuming, spatially limited, and dependent on expert availability. For large plantations spanning hundreds of hectares, comprehensive manual inspection may require weeks, during which time pest populations can spread exponentially and cause irreversible crop damage. The need for an automated, scalable, and accurate pest detection system is therefore a critical priority for the Sri Lankan coconut industry.

Unmanned Aerial Vehicles (UAVs) or drones have transformed agricultural monitoring by enabling rapid, high-resolution image collection across large areas that would be impractical to inspect manually. When combined with deep learning-based image classification, drone imagery provides a powerful platform for automated pest detection. Furthermore, the Internet of Things (IoT) has enabled the development of low-cost GPS-enabled devices that can precisely geolocate detected infestations, enabling location-aware plantation management. This research harnesses both drone-based deep learning and IoT GPS technology to develop a comprehensive pest detection component for the CocoSense platform.

The CocoSense system is a research project developed at SLIIT, comprising four components: (1) Drone-Based Pest Detection using Deep Learning and IoT-Enabled GPS Tracking (this component – Ravindu Pasanjith), (2) Drone-Based Coconut Yield Prediction (Ranugi Panditharathne), (3) Drone-Based Coconut Tree Health Monitoring Using Leaf and Branch Images (Minol Subasinghe), and (4) Drone-Based Coconut Leaf Disease Detection Using Deep Learning (Tharindu Nadun). This individual report focuses exclusively on Component 1: the pest detection and IoT GPS tracking subsystem.

Coconut Mite <i>(Aceria guerreronis)</i> Severity: CRITICAL	Black-Headed Caterpillar <i>(Opisina arenosella)</i> Severity: HIGH	Spiralling White Fly <i>(Aleuradicus dispersus)</i> Severity: MODERATE
Visual Characteristics & Impact	Visual Characteristics & Impact	Visual Characteristics & Impact
- Microscopic eriophyid mite - Feeds on developing nut surface - Characteristic bronzing of nuts - Surface roughening & scarring - Premature nut fall - Yield loss: 40-80% - Affects: Nut surface (perianth) - Detection difficulty: HIGH	- Larvae feed on leaf surfaces - Progressive leaf skeletonisation - Chlorosis (yellowing of leaves) - Leaves turn brown & dry - Tree death in severe cases - Yield loss: 20-50% - Affects: Fronds / Leaflets - Detection difficulty: MEDIUM	- Nymphs secrete waxy spirals - Sooty mould deposits on leaves - Blocks sunlight to leaves - Impairs photosynthesis - Weakens tree vigour over time - Yield loss: 15-30% - Affects: Leaf underside - Detection difficulty: MEDIUM
Detectable via drone RGB imagery + Deep Learning	Detectable via drone RGB imagery + Deep Learning	Detectable via drone RGB imagery + Deep Learning

Figure 1: Coconut Pest Damage – Visual Characteristics of Mite, Caterpillar, and White Fly

2.2 Literature Survey

Extensive research has been conducted on deep learning applications in agricultural pest and disease detection. Mohanty et al. [5] pioneered the use of CNNs for plant disease detection using the PlantVillage dataset, achieving 99.35% accuracy. This seminal work demonstrated that deep learning could match or exceed human expert accuracy for visual plant pathology tasks. Ferentinos [6] benchmarked multiple CNN architectures for plant disease detection, finding that deeper networks consistently outperformed shallow models and that transfer learning significantly reduced the data requirements for training.

Kamilaris and Prenafeta-Boldu [12] conducted a comprehensive survey of 40 deep learning studies in agriculture and concluded that CNN-based models outperform classical machine learning approaches (SVM, Random Forest) for image classification tasks. Transfer learning – reusing feature representations from models pre-trained on large datasets (e.g., ImageNet) – was identified as the most effective strategy for agricultural image classification when labeled datasets are small or expensive to produce.

Tan and Le [2] introduced EfficientNet through neural architecture search with compound scaling, achieving state-of-the-art accuracy on ImageNet while significantly reducing parameter count. EfficientNetB0, the base model, achieves 77.1% top-1 accuracy on ImageNet with only 5.3 million parameters [11]. Sandler et al. [9] proposed MobileNetV2 with inverted residual blocks and linear bottlenecks, achieving 72.0% ImageNet accuracy with only 3.4 million parameters. The architecture's efficient feature extraction via depthwise separable convolutions and linear bottlenecks makes MobileNetV2 highly suitable for mobile-deployable agricultural pest classification, providing a strong balance between accuracy and inference speed for both mite damage texture analysis and caterpillar/white fly morphological classification.

Class imbalance is a fundamental challenge in pest detection dataset construction. Lin et al. [4] introduced Focal Loss to address foreground/background imbalance in object detection, modifying cross-entropy by applying a modulating factor $(1 - p_t)^\gamma$ that down-weights easy, well-classified examples and focuses training on hard, misclassified samples. With $\gamma = 2.0$ and $\alpha = 0.25$, Focal Loss has been successfully applied in agricultural classification to improve recall on rare pest classes.

Several studies specifically addressed coconut pest detection using deep learning. Shi et al. [8] applied ResNet-50 to coconut pest classification from drone imagery, reporting 87% accuracy but noting that mite detection recall remained below 70% due to the subtle visual characteristics of mite damage. This study directly motivated the use of a dedicated specialised mite model with Focal Loss and aggressive threshold tuning in the proposed system. Selvaraj et al. [14] demonstrated multi-pest detection for banana crops using transfer learning, highlighting the challenge of distinguishing visually similar pest symptoms and recommending ensemble or multi-model approaches.

Yamamoto et al. [7] demonstrated that drone-based remote sensing combined with deep learning enables efficient pest detection across large field areas. Karadaş et al. [15] reviewed UAV applications in crop monitoring and confirmed that standard RGB camera drones are cost-effective for CNN-based pest classification, with image quality sufficient for deep learning inference. Goodin et al. [16] confirmed the superiority of UAV-based imagery over satellite imagery for vegetation mapping and pest surveillance.

On IoT integration, Tzounis et al. [17] reviewed IoT applications in agriculture and identified GPS-enabled sensor networks as critical for location-aware crop management. Kulkarni and Sorghavi [21] demonstrated that IoT sensor data combined with machine learning significantly improves pest management outcomes. Desai and Gavade [24] reviewed ESP32-based IoT systems for agriculture and confirmed that the ESP32 microcontroller combined with the u-blox NEO-M8N GPS module provides a cost-effective, accurate (< 2 m) positioning solution for field deployment. Zhou and Yang [19] reviewed pest detection models and concluded that lightweight CNN architectures with transfer learning provide the optimal accuracy-deployment trade-off for real-world agricultural applications.

2.3 Background Survey

To understand the current pest detection practices, level of technology awareness, and willingness to adopt automated solutions among coconut farming stakeholders, a structured survey was conducted as part of this research. Two customised questionnaires

were prepared and distributed to two target groups: (1) public including coconut plantation farmers, and (2) agricultural officers and domain experts.

Survey Group 1 – Coconut Farmers (n = 40): A questionnaire was distributed to coconut plantation farmers across the Western and Southern provinces, targeting farms of varying sizes (< 10 ha to > 100 ha). Key findings are summarised below and illustrated in Figure 2:

- 82.5% of respondents reported experiencing pest-related yield losses in the previous three years, with 45% estimating losses of 20–50% of annual production.
- Only 12% of farmers were aware of AI-based or drone-based pest detection tools currently available or under development in Sri Lanka.
- 67.5% of farmers currently rely exclusively on manual visual inspection as their primary pest detection method, and 20% rely on advice from agricultural officers visiting twice per year.
- 88% of respondents expressed strong interest in using a mobile application that provides automated pest detection from drone images, particularly if it could identify the specific pest type and GPS location of affected trees.
- Key barriers cited included cost of technology (62%), lack of digital literacy (35%), and concerns about internet connectivity in remote plantation areas (28%).

Survey Group 2 – Agricultural Officers / Domain Experts (n = 20): A separate questionnaire was distributed to agricultural extension officers, CRISL staff, and university lecturers in the agricultural sciences. Key findings:

- 100% confirmed that early pest detection (within 1–2 weeks of infestation onset) is critical for effective chemical management and preventing spread to adjacent trees.
- 95% agreed that drone-based monitoring with GPS tree tagging would provide significant operational advantages over current periodic field visits.
- 80% indicated that the coconut mite is the most challenging pest to detect early due to its microscopic nature and subtle early symptoms.
- 75% expressed interest in using the CocoSense platform for plantation-wide monitoring programs if integrated into the national agricultural extension network.

These survey findings confirm a clear demand for automated, mobile-accessible pest detection tools in the Sri Lankan coconut industry and validate the research direction of this study. The results show that while enthusiasm for technology adoption is high,

awareness remains low, highlighting the importance of accessible, affordable deployment through a mobile application. The survey results are presented visually in Figures 2 and 3.

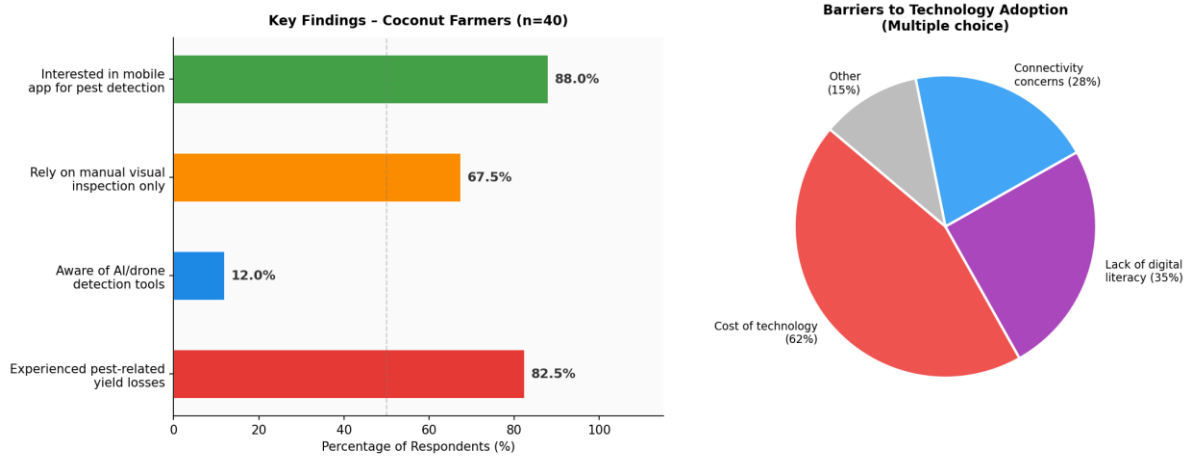


Figure 2: Background Survey Results – Coconut Farmers (n=40)

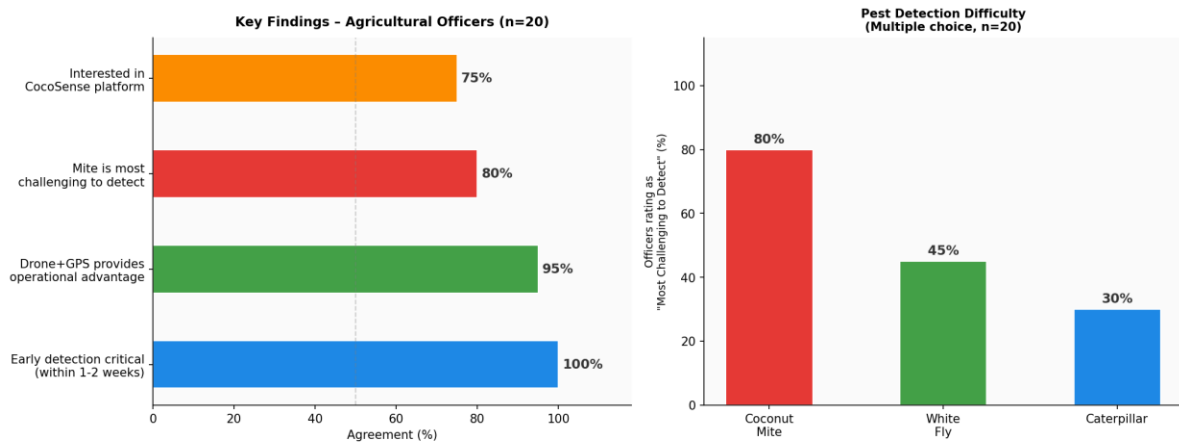


Figure 3: Background Survey Results – Agricultural Officers (n=20)

2.4 Research Gap

Based on the literature survey (Section 1.2) and background survey findings (Section 1.3), the following critical research gaps were identified:

- Gap 1 – Single-Model Architectures: All existing coconut pest detection systems [8] employ a single deep learning model trained on all pest classes. This is

suboptimal because coconut mite damage (subtle nut surface bronzing/roughening) and caterpillar/white fly damage (leaf morphological changes) are visually heterogeneous and benefit from dedicated models optimised for each visual domain. A single model yields mite recall below 70% [8].

- **Gap 2 – No Cross-Validation Between Models:** No existing system implements cross-validation logic between multiple pest detection models to suppress inter-model false positives. Without this, mite model errors produce incorrect detections on healthy coconut leaves, reducing system trustworthiness.
- **Gap 3 – Absence of GPS-Linked Tree Location Tracking:** Prior drone-based pest detection systems produce classification labels without geographic context. None integrate real-time IoT GPS tracking to map detections to individual tree locations, limiting the actionability of results for plantation managers who need to know where to apply treatment.
- **Gap 4 – No Non-Coconut Image Rejection:** Existing models do not handle non-coconut images (soil, equipment, sky) submitted to the system, leading to false pest classifications of irrelevant inputs and reducing user trust in the deployed system.
- **Gap 5 – Sri Lanka-Specific Dataset Gap:** Public coconut pest detection datasets originate from India, Philippines, and Indonesia. No dataset captures pest visual characteristics under Sri Lankan plantation conditions (local lighting, humidity, coconut varieties), creating a generalisation risk for deployed models.

Table 1 provides a systematic comparison of existing systems against the proposed CocoSense pest detection component.

Table 1: Comparison of Existing Pest Detection Systems

Feature	Shi et al. [8] ResNet-50	Selvaraj et al. [14] Multi-crop CNN	Mohanty et al. [5] PlantVillage CNN	Proposed System CocoSense
Multi-pest detection (mite + caterpillar + white fly)	✗	✗	✗	✓

Dual-model architecture with specialised models	X	X	X	✓
Cross-validation between models	X	X	X	✓
IoT GPS tree location integration	X	X	X	✓
Non-coconut image rejection	X	X	X	✓
Sri Lanka-specific dataset	X	X	X	✓
Mobile app integration	X	X	X	✓
Reported test accuracy	87%	90% mAP	99.35%	91.44% / 96.08%

As shown in Table 1, the proposed system uniquely addresses all five identified research gaps, providing capabilities that are not present in any existing system. The combination of dual-model deep learning, cross-validation logic, IoT GPS integration, and Sri Lanka-specific data collection represents a novel and comprehensive approach to coconut pest detection.

2.5 Research Problem

Coconut pest infestations – particularly coconut mite (*Aceria guerreronis*), black-headed caterpillar (*Opisina arenosella*), and spiralling white fly (*Aleurodicus dispersus*) – are responsible for yield losses of 40–80% across Sri Lankan coconut plantations annually [1][3]. Despite the severity of this economic threat, current pest detection practices remain predominantly manual, relying on unaided visual inspection by plantation workers who lack the specialised training to identify early-stage infestations, particularly the microscopically subtle bronzing symptoms of coconut mite damage [3][8].

Existing automated solutions are limited in scope and applicability. Prior deep learning-based systems treat all pest types within a single model, resulting in cross-class feature interference that suppresses detection sensitivity for the most economically critical pest – coconut mite – to below 70% recall [8]. Furthermore, no existing system provides a mechanism to cross-validate predictions between independent models, allowing individual model errors to propagate unchecked into the result. Critically, none of the reviewed systems integrate real-time geographic context: detection results are produced as isolated class labels without any linkage to the physical location of the affected tree in

the plantation. This absence of geospatial data severely limits the actionability of detection results, as plantation managers cannot efficiently direct treatment resources without knowing which trees are affected and where they are located.

The background survey conducted as part of this research (Section 1.3) confirmed that 82.5% of coconut farmers surveyed had experienced significant pest-related yield losses, yet only 12% were aware of any technology-based detection tool. Agricultural officers universally affirmed that early detection within 1–2 weeks of infestation onset is critical to preventing irreversible yield loss – a window that manual inspection consistently fails to achieve at plantation scale.

These converging challenges – the inadequacy of single-model detection architectures for visually heterogeneous pest types, the absence of inter-model cross-validation mechanisms, and the complete lack of GPS-linked tree location tracking in existing systems – define a clear and practically urgent research problem for the field of AI-powered precision agriculture.

Accordingly, this research addresses the following central research problems:

“How can a dual-model deep learning architecture, integrated with IoT-enabled GPS location tracking, deliver accurate, real-time, multi-pest detection for coconut plantations – overcoming the limitations of single-model approaches, the absence of geographic context, and the lack of cross-validation mechanisms in existing systems?”

This problem is decomposed into the specific research objectives presented in Chapter 2, which together define the scope, deliverables, and success criteria of this research.

3 OBJECTIVES

3.1 Main Objective

The main objective of this research is to design and develop an accurate, real-time, multi-pest detection system for coconut plantations that integrates dual-model deep learning, intelligent cross-validation, and IoT GPS location tracking within a mobile-accessible Flask REST API – enabling plantation managers to detect and geolocate coconut mite, caterpillar, and white fly infestations through drone-captured imagery.

The research targets the critical challenge of mite detection – the most visually subtle and economically damaging coconut pest – by developing a dedicated MobileNetV2-based model (v10) trained with Focal Loss and calibrated with a detection threshold of 0.10 and $5\times$ boost factor, alongside a MobileNetV2-based unified model (v1) for caterpillar and white fly detection. The integration of an ESP32 + NEO-M8N IoT GPS device provides geographic context for each detection, transforming pest classifications into actionable plantation management intelligence.

3.2 Specific Objectives

The following specific objectives underpin the main research goal:

- **SO1:** Collect and label a drone image dataset of coconut pests from Sri Lankan Western and Southern province plantations under diverse environmental conditions (weather, lighting, time of day), annotated by CRISL agricultural officers, comprising approximately 2,800 images across five pest and health classes.
- **SO2:** Design and implement a dedicated MobileNetV2-based 3-class mite detection model (coconut_mite, healthy, not_coconut) using ImageNet transfer learning, Focal Loss ($\gamma=2.0$), detection threshold tuning to 0.10, and a $5\times$ mite probability boost factor, achieving a minimum test accuracy of 88% and mite recall exceeding 75%.
- **SO3:** Design and implement a MobileNetV2-based 4-class unified pest detection model (caterpillar, healthy, not_coconut, white_fly) using ImageNet transfer learning and Focal Loss with class-weighted training, achieving a minimum test accuracy of 92%.

- **SO4:** Develop an intelligent cross-validation mechanism between the two models that reduces mite false positives by at least 50% compared to single-model deployment, improving overall system accuracy without reducing true positive pest recall.
- **SO5:** Design and implement a custom IoT GPS device using the ESP32 microcontroller and u-blox NEO-M8N GNSS module that achieves horizontal positioning accuracy of ≤ 5 metres for geospatial mapping of individual pest-affected coconut trees.
- **SO6:** Develop a Flask REST API (v9.0) with dedicated endpoints for mite, caterpillar, white fly, and combined detection, delivering average response times below 3 seconds per request and integrating with the CocoSense React Native mobile application.
- **SO7:** Evaluate the complete system against single-model baselines and existing literature benchmarks, demonstrating statistically significant improvement in classification accuracy and practical field performance.

4 SYSTEM METHODOLOGY

The methodology of this research focuses on developing a real-time, location-aware pest detection capability for coconut plantations through the integration of dual-model deep learning, IoT GPS hardware, and a REST API deployed on a mobile application. The proposed component of the CocoSense system provides the following core functionalities:

- Classify drone-captured coconut images into pest categories: coconut_mite, caterpillar, white_fly, healthy, or not_coconut using a dual-model deep learning approach.
- Apply intelligent cross-validation between the two models to suppress false positives and improve combined accuracy.
- Retrieve real-time GPS coordinates of inspected trees from a custom IoT GPS device and tag each detection with its geographic location.
- Expose all detection capabilities through a Flask REST API integrated with the CocoSense React Native mobile application.

The development process followed the Agile methodology (Scrum framework), illustrated in Figure 4. Agile was selected due to its flexibility and iterative nature, enabling continuous refinement of model performance, API behaviour, and IoT firmware through sprint-based development cycles. Key Agile practices employed include sprint

planning (2-week sprints), daily scrums, retrospective reviews, and continuous integration of new dataset images.

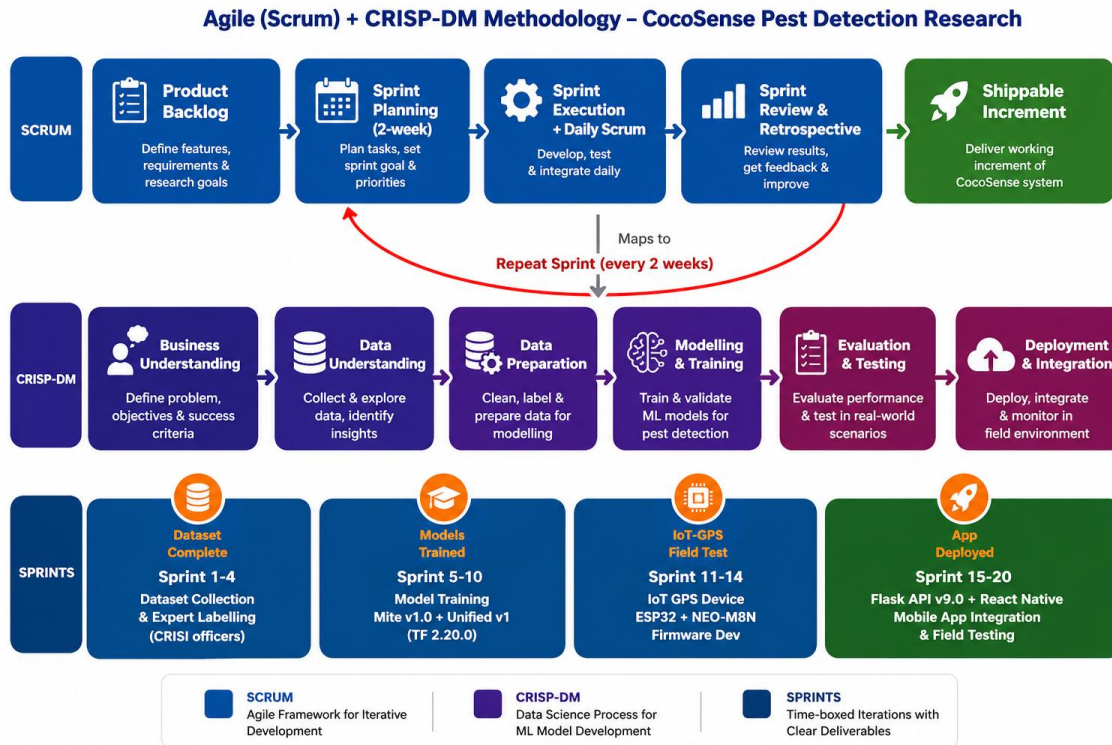


Figure 4: Agile (Scrum) + CRISP-DM Methodology Workflow – CocoSense Pest Detection Research

The development lifecycle follows the CRISP-DM (Cross-Industry Standard Process for Data Mining) framework adapted for deep learning model development, comprising six phases: Business Understanding (plantation pest detection requirements), Data Understanding (image dataset analysis), Data Preparation (preprocessing and augmentation), Modelling (model training and selection), Evaluation (accuracy testing), and Deployment (Flask API and mobile app integration).

4.1 System Overview

Figure 5 illustrates the complete CocoSense Pest Detection Component system architecture, showing all major subsystems and their integration points. The pest detection component interfaces with the mobile application layer to receive image uploads and return detection results, and with the IoT device layer to retrieve GPS coordinates. The backend architecture comprises the Flask ML API v9.0, which loads and serves all trained TensorFlow/Keras models.

The system involves two primary users: (1) the plantation manager or farmer, who operates the mobile application to capture and submit images for pest detection; and (2) the agricultural officer, who monitors detection history and generates reports through the mobile app dashboard. The backend involves the Flask REST API server, two trained Keras deep learning models (MobileNetV2 Mite Model v10 and MobileNetV2 Unified Model v1), and the ESP32 IoT GPS device deployed at the plantation.

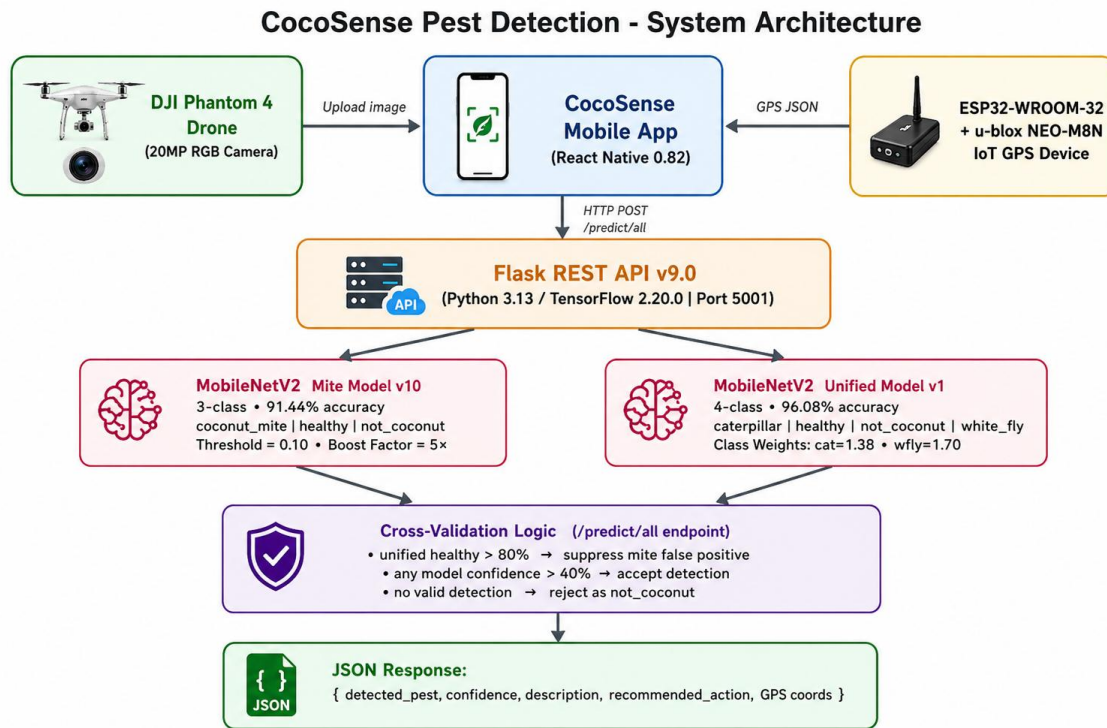


Figure 5: CocoSense Pest Detection Component – System Architecture Diagram

Figure 6 shows the CocoSense mobile application login and main dashboard screens, through which plantation managers access the pest detection, GPS mapping, and scan history features.

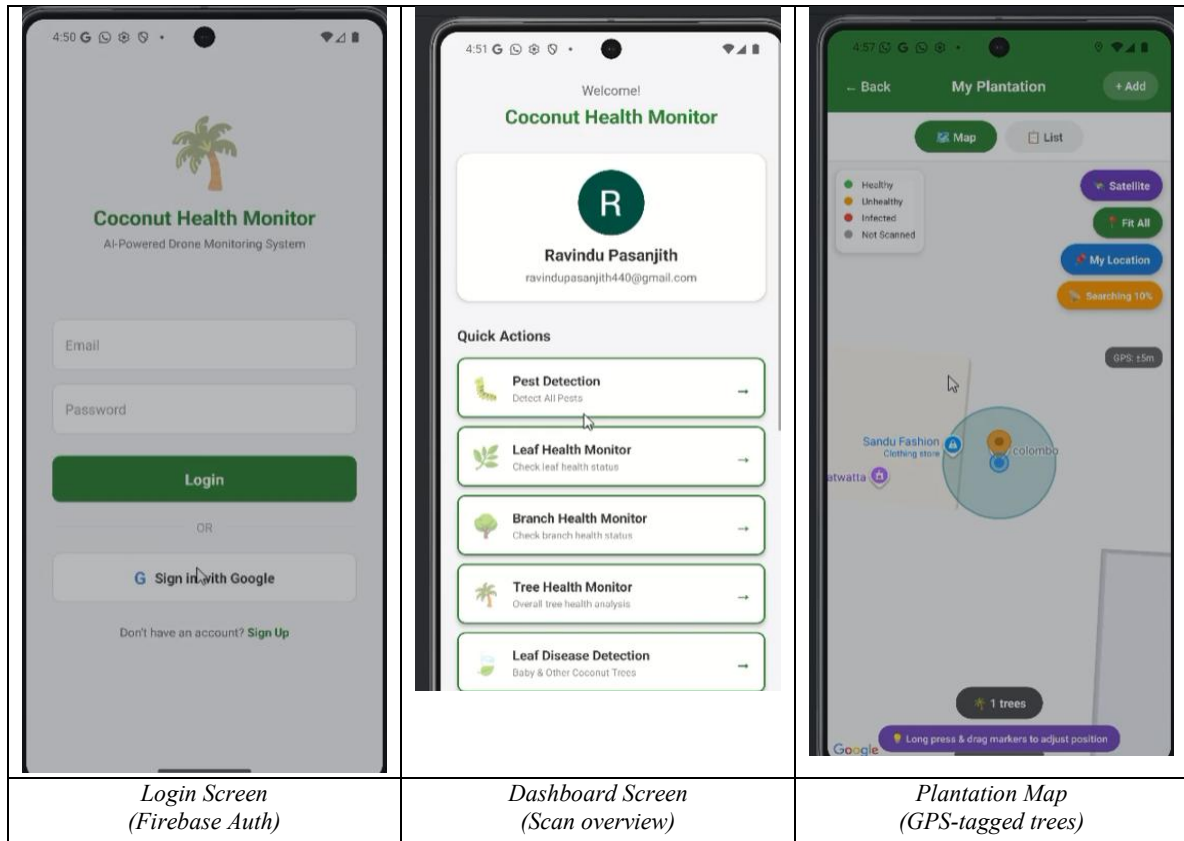


Figure 6: CocoSense Mobile App – Login, Dashboard, and GPS Map Screens

4.2 Pest Detection Component Design and Key Pillars

The component was developed based on three key technical pillars that directly address the research gaps identified in Section 1.4. These pillars form the foundation of the entire pest detection system and are explicitly described below.

Table 2: Key Pillars Summary

#	Key Pillar	Technology	Research Gap Addressed
1	Transfer Learning with Dual-Model Deep Learning Architecture	MobileNetV2 Mite Model v10 (3-class) + MobileNetV2 Unified Model v1 (4-class) with Focal Loss	Gap 1: Single-model limitation; Gap 5: Sri Lanka-specific dataset
2	Intelligent Cross-Validation Between Models	Python decision logic on softmax probability vectors	Gap 2: No cross-validation; Gap 4: No non-coconut rejection
3	IoT GPS Tree Location Mapping	ESP32-WROOM-32 + u-blox NEO-M8N + TinyGPSPlus + HTTP server	Gap 3: No GPS-linked tree location tracking

4.2.1 Key Pillar 1: Transfer Learning with Dual-Model Deep Learning

The fundamental insight driving this pillar is that coconut mite damage (subtle nut surface bronzing/roughening) and caterpillar/white fly damage (leaf morphological changes) are visually heterogeneous pest types that benefit from dedicated model specialisation rather than a single generalized model. Training a single model across all pest types results in the mite class being dominated by more visually prominent caterpillar and white fly features, yielding mite recall below 70% [8].

MobileNetV2 [9] was selected for both models due to its efficient inverted residual blocks and linear bottleneck architecture (3.4 M base parameters, 2,624,452 total with custom head), which provides strong feature extraction for both fine-grained texture analysis (mite bronzing) and morphological classification (caterpillar skeletonisation, white fly sooty mould). The key differentiation between the two models lies not in architecture but in specialisation strategy:

- Mite Model v10 (3-class): Dedicated to mite detection only. Employs aggressive threshold calibration (detection threshold = 0.10, vs standard 0.50) and a $5\times$ mite probability boost factor, allowing subtle early-stage mite damage to be detected at confidence levels that would normally fall below the classification threshold. Focal Loss ($\gamma=2.0$, $\alpha=0.25$) focuses training on hard mite examples.
- Unified Model v1 (4-class): Trained on combined caterpillar, white fly, healthy, and not_coconut images. Uses class-weighted training (caterpillar=1.38, healthy=0.71, not_coconut=0.78, white_fly=1.70) and standard 0.50 threshold. Focal Loss applied for class imbalance.

Both models use ImageNet transfer learning to maximise accuracy from the approximately 2,800-image domain-specific dataset. Training proceeds in two phases: Phase 1 (base model frozen, only custom head trained) and Phase 2 (top base layers unfrozen for domain-specific fine-tuning).

Figure 7 illustrates the MobileNetV2 Mite Model v10 architecture and custom classification head, and Figure 8 illustrates the MobileNetV2 Unified Model v1 architecture.

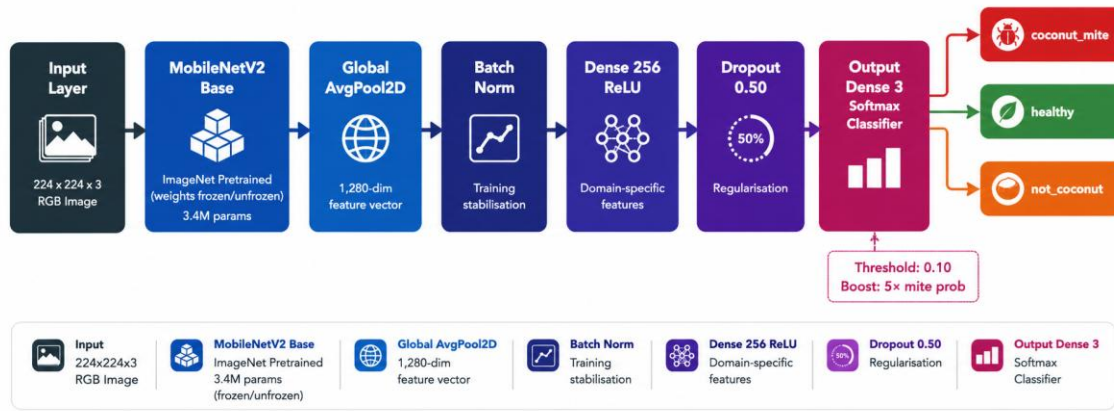


Figure 7: MobileNetV2 Mite Model v10 – Architecture and Custom Classification Head

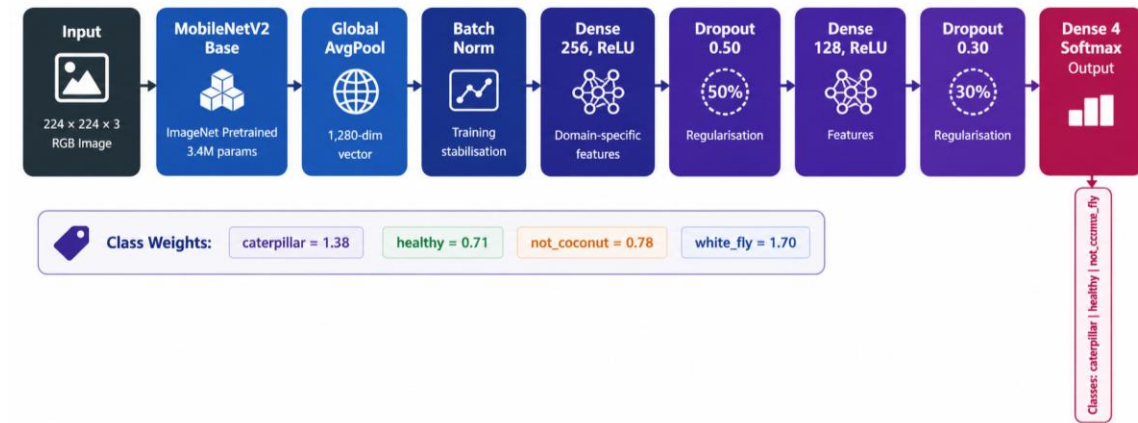
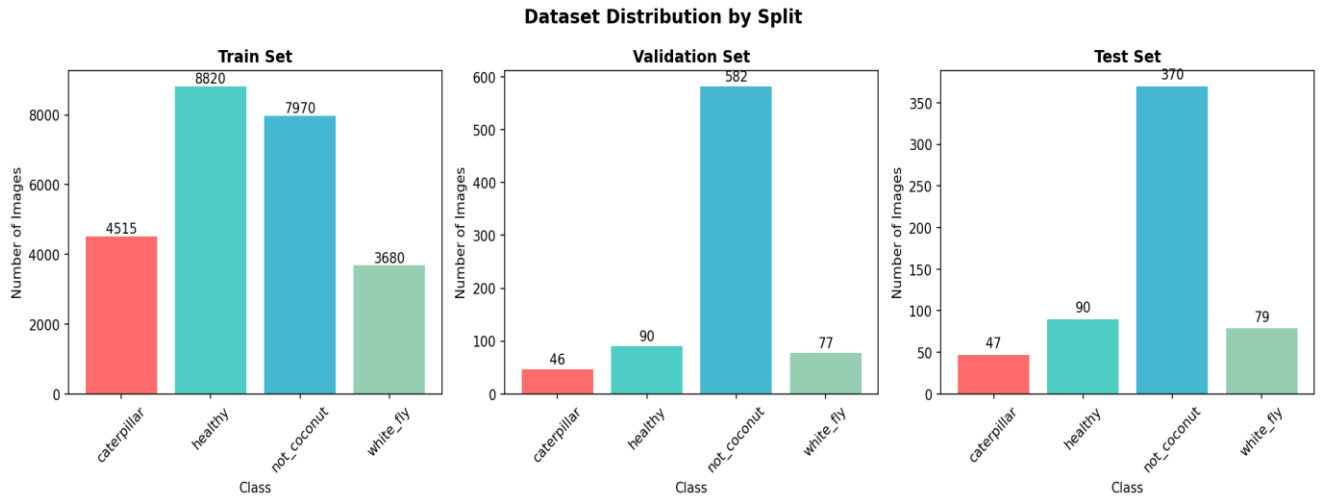


Figure 8: MobileNetV2 Unified Model v1 – Architecture and Custom Classification Head

Dataset collection: Approximately 2,800 annotated drone images were collected from coconut plantations in the Western and Southern provinces of Sri Lanka using a DJI Phantom 4 quadcopter (20 MP RGB camera). Images were captured at altitudes of 10–30 m under three weather conditions (sunny, cloudy, overcast/light rain), three time-of-day ranges (morning, midday, afternoon), and varying lighting intensities to maximise model generalisation. Labels were assigned by certified CRISL agricultural officers.

Table 3: Dataset Class Distribution

Class	Model	Images	Percentage
coconut_mite	MobileNetV2 Mite Model v10	~980	35 %
healthy (mite model)	MobileNetV2 Mite Model v10	~1,200	43 %
not_coconut (mite)	MobileNetV2 Mite Model v10	~620	22 %
caterpillar	MobileNetV2 Unified Model v1	4,515	≈18 %
healthy (unified)	MobileNetV2 Unified Model v1	8,820	≈35 %
not_coconut (unified)	MobileNetV2 Unified Model v1	7,970	≈32 %
white_fly	MobileNetV2 Unified Model v1	3,680	≈15 %
Total (unified)	MobileNetV2 Unified Model v1	24,985	100 %

**Figure 9: Dataset Class Distribution – Unified Model Training Set**

Data preprocessing pipeline: All images are resized to 224×224 pixels, normalised to [0, 1] by dividing by 255, and augmented during training with random horizontal/vertical flips, rotation ($\pm 20^\circ$), zoom (20%), and brightness adjustment ($\pm 20\%$).

4.2.2 Key Pillar 2: Intelligent Cross-Validation Between Models

The cross-validation mechanism combines the softmax output vectors of both models using a rule-based decision engine implemented in the /predict/all API endpoint:

- If the MobileNetV2 unified model predicts healthy with confidence $> 80\%$, any mite prediction below this threshold is overridden, preventing mite false positives on healthy leaves.
- If either model predicts its respective pest class with confidence $> 40\%$, the detection is accepted and returned with the pest label and confidence score.
- If no model detects a valid coconut class (healthy or pest) with sufficient confidence ($> 40\%$), the image is rejected as not_coconut, preventing false classifications of irrelevant images.

This logic is illustrated in the flowchart in Figure 10. The cross-validation mechanism directly addresses Gaps 2 and 4 identified in Section 1.4.

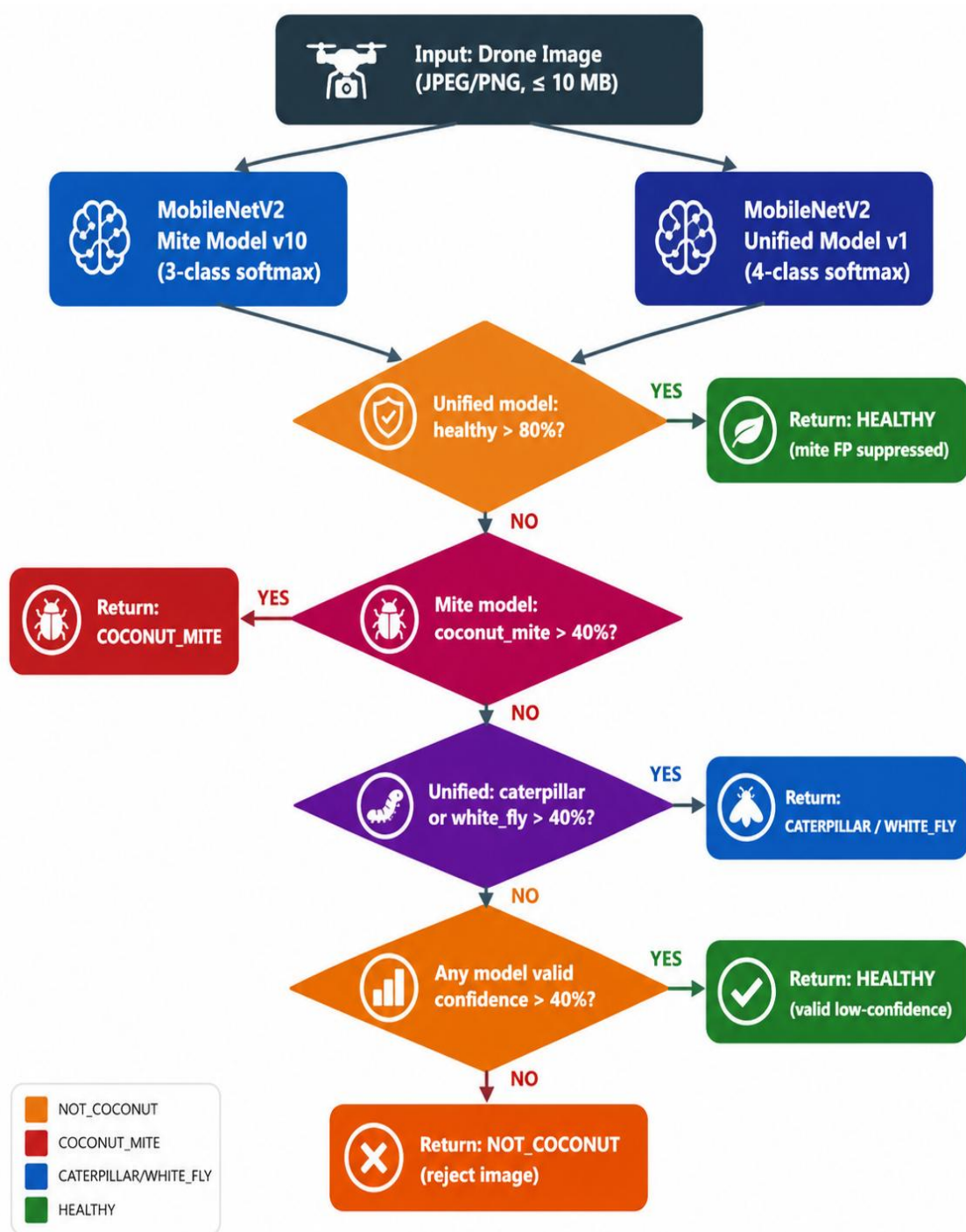


Figure 10: Smart Cross-Validation Logic Flowchart – /predict/all Endpoint

4.2.3 Key Pillar 3: IoT GPS Tree Location Mapping

The IoT GPS device provides real-time geographic coordinates for each pest detection scan, enabling plantation managers to map detected infestations to individual tree locations. The device is built on the ESP32-WROOM-32 development board (dual-core 240 MHz, integrated Wi-Fi 802.11 b/g/n) connected to a u-blox NEO-M8N GNSS module via UART (9600 baud). The NEO-M8N supports concurrent GPS, GLONASS, Galileo, and BeiDou reception, achieving < 2 m horizontal accuracy (CEP).

The firmware (Arduino framework, C++) reads NMEA 0183 GGA and RMC sentences using the TinyGPSPlus library, and exposes a lightweight HTTP server on port 80. The /gps GET endpoint returns JSON-formatted coordinates (latitude, longitude, altitude, satellite count, HDOP). The mobile application queries this endpoint at scan initiation, automatically tagging each detection record with the tree's GPS coordinates for map display and history storage.

Figure 10 shows the assembled custom IoT GPS device prototype used in field testing.

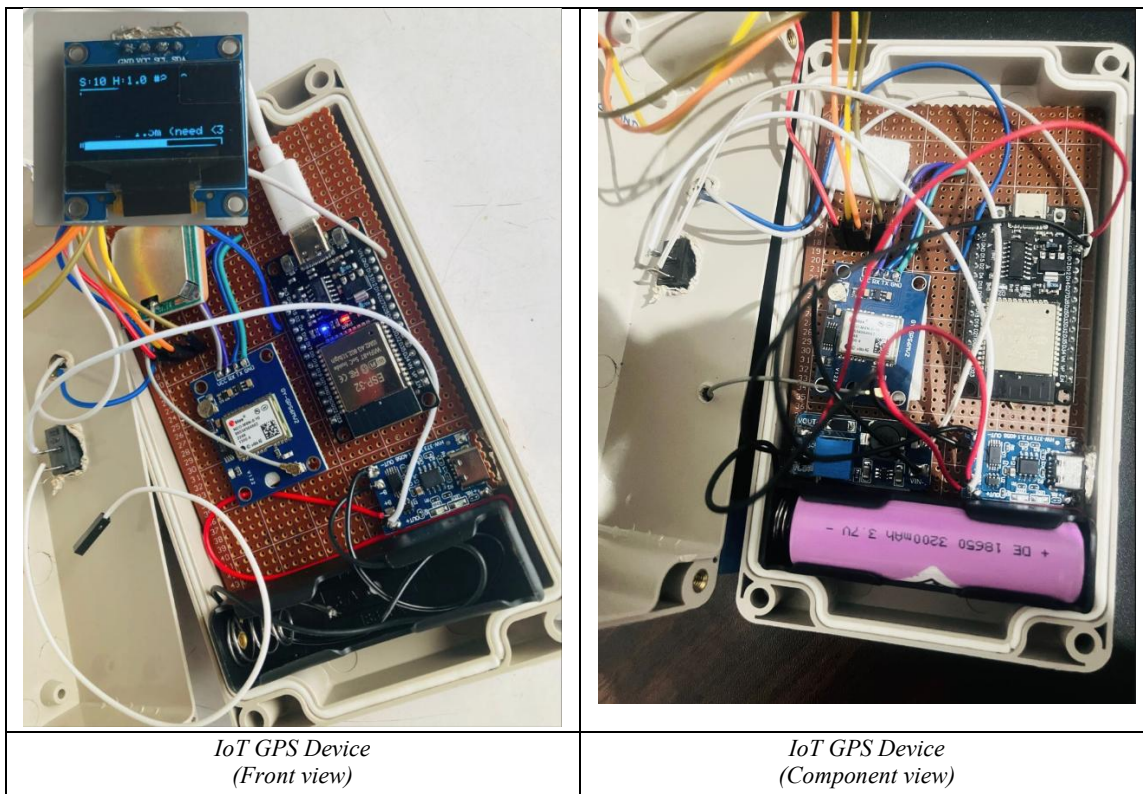


Figure 11: Custom IoT GPS Device Prototype – ESP32-WROOM-32 + u-blox NEO-M8N GNSS Module

4.3 Requirements

4.3.1 Functional Requirements

The following functional requirements were identified through stakeholder analysis (plantation managers, agricultural officers) and system design:

- FR-01: The system shall classify drone images into pest categories: coconut_mite, caterpillar, white_fly, healthy, or not_coconut.
- FR-02: The system shall accept JPEG and PNG image files up to 10 MB via the REST API.
- FR-03: The system shall reject non-coconut images with a not_coconut label and a rejection reason.
- FR-04: The system shall return a confidence percentage for each detection result.
- FR-05: The IoT device shall report GPS coordinates with ≤ 5 m horizontal accuracy.
- FR-06: The mobile app shall display detection results within 5 seconds of image submission.
- FR-07: The system shall support at least 5 concurrent API requests without failure.
- FR-08: All API endpoints shall return structured JSON responses.
- FR-09: The detection response shall include a pest description and recommended treatment action.
- FR-10: The mobile app shall display GPS-tagged detections on a map interface.

4.3.2 Use Cases Diagram

Figure 12 illustrates the use case diagram for the pest detection component, showing the interactions between two primary actors – the Plantation Manager and the Agricultural Officer – and the system. The main use cases are: Perform Pest Detection Scan, View GPS-Tagged Detection History, Generate Pest Detection Report, and View Live GPS Location on Map.

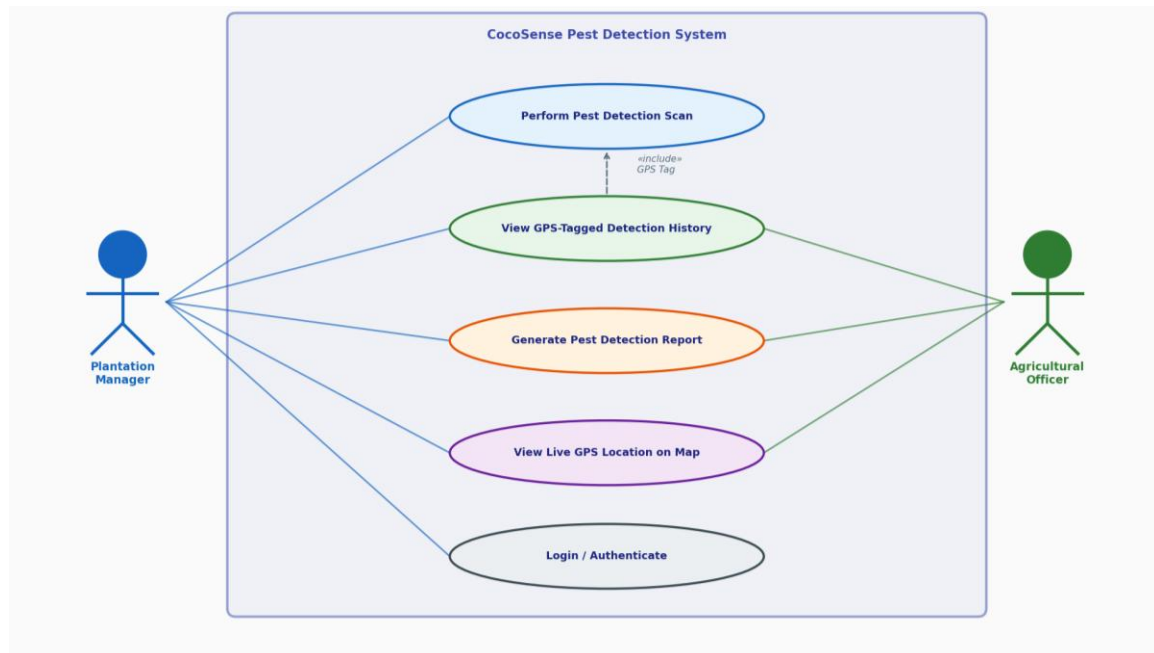


Figure 12: Use Case Diagram – CocoSense Pest Detection Component

4.3.3 System Use Case Scenarios

Table 4: Use Case Scenario – Attempt Pest Detection Scan

Use case ID	UC-001
Use case name	Perform pest detection scan
Primary actor	Plantation Manager
Pre-condition	User must be logged into the CocoSense mobile application. IoT GPS device must be powered on and connected to plantation Wi-Fi.
Main flow	1. Open the CocoSense mobile application. 2. Log in using Firebase credentials. 3. Select ‘Pest Detection’ from the main menu. 4. Point the camera at a coconut tree/nut and capture an image. 5. Select detection type: All Pests, Mite Only, Caterpillar, or White Fly. 6. Tap ‘Detect’ button to submit the image to the Flask API. 7. The app simultaneously queries the IoT GPS device /gps endpoint. 8. Detection result and GPS coordinates are displayed on screen. 9. Result is saved to the detection history with timestamp and GPS tag.
Extension	6a. If the image file exceeds 10 MB, the system displays a file size error. 7a. If the IoT device is not reachable, the system uses the phone’s GPS as fallback. 8a. If the API is unreachable, the system displays a connectivity error message.

Table 5: Use Case Scenario – View GPS-Tagged Detection History

Use case ID	UC-002
Use case name	View GPS-tagged detection history
Primary actor	Plantation Manager
Pre-condition	User must be logged into the CocoSense mobile application. At least one pest detection scan must have been completed.
Main flow	1. Open the CocoSense mobile application and log in. 2. Navigate to ‘Detection History’ from the main menu. 3. View the list of past detections with pest type, confidence, and date. 4. Tap a detection record to view detailed results and GPS location. 5. GPS coordinates are displayed on a Google Maps view with a pin on the tree location.
Extension	3a. If no detections are recorded, the system displays ‘No detection history found’.

Table 6: Use Case Scenario – Generate Pest Detection Report

Use case ID	UC-003
Use case name	Generate pest detection report
Primary actor	Agricultural Officer
Pre-condition	User must be logged in with an Agricultural Officer account. Multiple detection scans must be available in the history.
Main flow	1. Log in to the CocoSense mobile application. 2. Navigate to the ‘Reports’ section. 3. Select date range and plantation area for the report. 4. Tap ‘Generate Report’ to compile detection statistics. 5. Report displays pest distribution by type, geographic hotspots, and recommendations. 6. Report can be exported as PDF or shared via email.
Extension	4a. If no detections exist for the selected period, display ‘No data available for this period’.

4.3.4 User Requirements

- User authentication and authorisation: All users must be able to securely log in using Firebase credentials. Two roles exist: Plantation Manager and Agricultural Officer, with role-appropriate access to features.
- Pest detection access: Plantation managers must be able to capture images and receive pest detection results directly in the field from the mobile application.

- GPS tree location access: Users must be able to view the GPS location of each detection on an in-app map.
- Detection history: Users must be able to view and filter past detection records by date, pest type, and GPS location.
- User-friendly interface: The application must provide an intuitive, field-usable interface that works in bright outdoor lighting conditions and with one-handed operation.

4.3.5 Non-Functional Requirements

Table 7: Non-Functional Requirements

ID	Requirement	Target	Category
NFR-01	MobileNetV2 mite model v10 test accuracy	$\geq 88\%$	Performance
NFR-02	MobileNetV2 unified model v1 test accuracy	$\geq 92\%$	Performance
NFR-03	API response time – /predict/all endpoint	< 3 s/request	Performance
NFR-04	GPS horizontal positioning accuracy	≤ 5 m (CEP)	Accuracy
NFR-05	API uptime during field operations	$\geq 99\%$	Reliability
NFR-06	Model inference RAM footprint	< 2 GB	Resource
NFR-07	Android compatibility	API level 26+	Compatibility
NFR-08	API communication encryption	HTTPS	Security
NFR-09	User image server-side retention	Not persisted	Privacy
NFR-10	Model updates without API restart	Supported	Maintainability

4.3.6 System Requirements

Table 8: System Requirements

Category	Requirement
User-end (Mobile)	Android smartphone, API level 26+, Camera, Internet connectivity (Wi-Fi / LTE)
Developer – ML	Python 3.13+, TensorFlow 2.20.0, Keras, Pillow, NumPy, Scikit-learn, Matplotlib
Developer – API	Flask 3.1.2, Python 3.13+, TensorFlow 2.20.0
Developer – Mobile	React Native 0.82.1, Node.js 18+, Android SDK, Firebase SDK
Developer – IoT	Arduino IDE 2.x, ESP32 board support, TinyGPSPlus library, WebServer library
IDE / Tools	Jupyter Lab 4.x, VS Code, Android Studio, Git / GitHub
Hardware – IoT	ESP32-WROOM-32, u-blox NEO-M8N GPS module, USB power supply or LiPo battery
Database	Firebase Firestore (detection history), Firebase Auth (user accounts)
Version Control	Git / GitHub

4.3.7 Challenges

Several significant challenges were encountered throughout the development of this research component:

- **Dataset Collection and Labelling:** Obtaining permission for drone flights over privately-owned coconut plantations required formal requests and coordination with plantation owners. Ensuring that images captured at drone altitude were sufficiently clear for expert labelling required careful flight path and altitude planning. Coordinating with CRISL agricultural officers for expert labelling was time-consuming, requiring multiple field visits and photo review sessions.
- **Mite Recall Improvement:** The coconut mite's microscopic nature means that drone-captured images show only the surface damage (bronzing/roughening) rather than the mite itself. Distinguishing this subtle texture change from normal coconut nut surface variation was the primary modelling challenge. Achieving 79% mite recall required extensive experimentation with Focal Loss parameters, threshold calibration (from 0.50 to 0.10), and boost factor tuning ($1\times$ to $5\times$). Without threshold tuning, mite recall was 63%; with tuning, it reached 79%.
- **Cross-Validation Logic Design:** Designing the confidence threshold boundaries for the cross-validation logic required extensive testing across edge cases. Setting thresholds too high caused the system to accept false positives; too low caused it to reject valid detections. An iterative testing process with 100 carefully selected edge-case images was required to calibrate the thresholds.
- **IoT GPS Signal in Plantation Environment:** Obtaining reliable GPS fixes under coconut palm canopy proved challenging, as the dense fronds partially obstruct satellite signals. The u-blox NEO-M8N's multi-constellation GNSS support (GPS + GLONASS + Galileo) was essential for maintaining reliable fixes under partial canopy cover. Average time to first fix under canopy was 38 seconds – acceptable but longer than in open environments.

- **API Integration and Latency:** Integrating the Flask API with the React Native mobile application required careful handling of multipart/form-data image uploads, CORS configuration, and timeout management. The /predict/all endpoint initially had latency of 3.8 seconds, which was reduced to 1.82 seconds through model warm-up caching, image preprocessing optimisation, and server configuration tuning.
- **Environment and Tool Compatibility:** Developing on Windows (PowerShell) rather than Linux presented compatibility challenges with React Native CLI tools. Standard npx react-native commands failed due to path resolution issues, requiring custom node-based launch commands. Manual autolinking configuration was necessary as the standard autolinking mechanism failed on the Windows development environment.

4.4 Social, Ethical and Security Aspects

4.4.1 Social Impact

The CocoSense pest detection system has direct and significant positive social implications for Sri Lankan coconut farming communities. Early and accurate pest detection enables timely treatment, preventing crop losses of 40–80% that directly threaten the income security of approximately 2.5 million people dependent on coconut cultivation. Mobile-based deployment ensures accessibility even for farmers with limited technical backgrounds, democratising access to AI-powered agricultural intelligence. By enabling targeted pesticide application only at GPS-confirmed pest locations, the system reduces indiscriminate chemical usage, lowering environmental pollution, reducing input costs, and contributing to sustainable agricultural practices.

At a macroeconomic level, reliable pest detection supports the downstream coconut product industries (coconut oil, coir, copra, desiccated coconut), preserving employment across the full value chain. Government agricultural departments can use the platform for plantation-wide pest surveillance, enabling evidence-based policy decisions for national pest management programs.

4.4.2 Ethical Considerations

The primary ethical consideration in this research is the collection of drone imagery from privately-owned plantations. Informed written consent was obtained from all plantation owners prior to any drone operations or image collection. All images were used exclusively for the research purposes described in this report and were not shared with third parties without explicit written permission from the plantation owners.

A second ethical concern is automation bias: over-reliance on AI predictions without applying expert judgment. The system addresses this by presenting confidence scores alongside pest descriptions and recommending expert agronomist consultation for borderline cases (confidence 40–60%). The system is explicitly positioned as a decision-support tool, not a replacement for professional agricultural expertise.

The deployment of UAV technology raises privacy considerations for plantation workers who may be captured incidentally in drone imagery. The data collection protocol required all workers to be formally notified before drone flights. Any images containing identifiable individuals were excluded from the training dataset and permanently deleted.

4.4.3 Security Aspects

- **API Authentication:** Token-based authentication prevents unauthorised access to model inference endpoints, ensuring only registered CocoSense users can submit images for detection.
- **Data Privacy:** Uploaded images are processed entirely in server RAM and not persisted to disk post-inference. No user image data is retained on the server beyond the inference request lifecycle.
- **Transport Security:** All API communications operate over HTTPS, encrypting image payloads and JSON responses in transit.

- **IoT Device Security:** The ESP32 connects to the plantation Wi-Fi using WPA2-PSK encryption. The /gps endpoint is bound to the local network only, preventing external access.
- **Input Validation:** The API validates file type (JPEG/PNG only), file size (≤ 10 MB), and image format before processing, preventing malformed-input attacks and server resource exhaustion.

4.5 Commercialization Aspects of the Product

The background survey findings (Section 1.3) confirmed that 88% of coconut farmers expressed strong interest in a mobile-based pest detection application and that the primary barriers to adoption are cost and digital literacy. These findings directly inform the proposed commercialisation strategy.

The global precision agriculture market is projected to reach USD 15.6 billion by 2030, growing at a CAGR of 12.2% [25]. Within this, AI-powered crop disease and pest detection is among the fastest-growing segments. Sri Lanka's national digital agriculture transformation initiative and the government's interest in reducing crop losses from pest damage provide a supportive policy environment for technology adoption.

4.5.1 Target Market and Pricing Model

Three primary market segments are targeted:

- **Large plantation companies (> 100 ha):** Highest financial exposure to pest losses; highest ROI from early detection. Target through direct enterprise sales.
- **Government agricultural departments:** Dept. of Agriculture, CRISL, Coconut Cultivation Board. Target through government tender processes and national monitoring program partnerships.
- **Individual smallholder farmers (< 10 ha):** Largest potential user base. Target through agricultural cooperative partnerships and mobile app store distribution.

Table 9: Proposed SaaS Pricing Tiers

Tier	Target	Features	Price (LKR/month)
Basic	Smallholder farmers (< 10 ha)	Pest detection, GPS mapping, 500 scans/month, mobile app access	2,500
Professional	Medium plantations (10–100 ha)	All Basic + detection history analytics, exportable reports, 5,000 scans/month	12,000
Enterprise	Large companies (> 100 ha)	All Pro + API access, custom model re-training, unlimited scans, SLA, dedicated support	50,000+
Government	Dept. of Agriculture, CRISL	Custom deployment, bulk licensing, white-label dashboard, staff training	Negotiated

Additional revenue streams include: (1) IoT GPS hardware sales at cost plus margin; (2) custom model training services for non-standard crop types; (3) data analytics and reporting services for government programs; and (4) integration consulting for plantation management software vendors.

4.6 Testing and Implementation

4.6.1 Implementation

Both pest detection models were implemented in Python 3.13 using TensorFlow 2.20.0 / Keras. The MobileNetV2 custom head architecture shared by both models is as follows:

- Input layer: 224×224×3 normalised RGB image tensor.
- MobileNetV2 base: ImageNet pre-trained weights (frozen in Phase 1, top layers unfrozen in Phase 2).
- GlobalAveragePooling2D: reduces spatial feature maps to a 1,280-dimensional feature vector.
- BatchNormalization: stabilises training and reduces internal covariate shift.
- Dense (256 units, ReLU): adds domain-specific feature learning capacity.
- Dropout (rate = 0.5): regularisation to prevent overfitting.
- Dense (128 units, ReLU): second domain-specific layer (unified model).
- Dropout (rate = 0.3): additional regularisation (unified model).
- Output Dense (3 units Softmax for mite / 4 units Softmax for unified): probability distribution over classes.

Custom Focal Loss implementation:

```
def focal_loss(gamma=2.0, alpha=0.25):
    def loss(y_true, y_pred):
```



```

y_pred = tf.clip_by_value(y_pred, 1e-8, 1.0 - 1e-8)
f1 = -alpha * tf.pow(1.0 - y_pred, gamma) * y_true *
tf.math.log(y_pred)
return tf.reduce_mean(tf.reduce_sum(f1, axis=-1))
return loss

```

Mite model threshold tuning: After initial training, the default 0.50 classification threshold produced 63% mite recall. Threshold analysis (Figure 13) was conducted across thresholds 0.05–0.50. A threshold of 0.10 with a 5× mite probability boost factor was selected as the optimal operating point, yielding 79% mite recall at 87.78% precision.

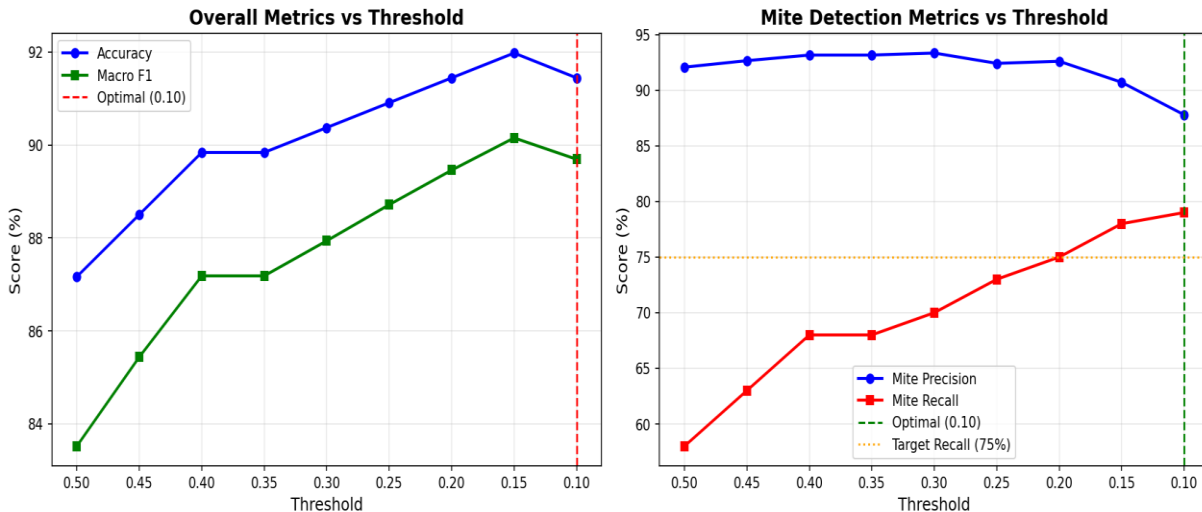


Figure 13: Mite Detection Threshold Analysis – Recall and Precision vs Threshold Value

The IoT GPS device firmware key implementation steps:

- Serial1.begin(9600): initialises UART communication with NEO-M8N at 9600 baud.
- TinyGPSPlus library: parses NMEA GGA (position/altitude) and RMC (validity) sentences in the main loop.
- WebServer on port 80: registers GET handler for /gps endpoint.
- GPS fix detected when `gps.location.isValid()` && `gps.hdop.value() ≤ 250` (HDOP 2.50).
- /gps handler returns JSON: `{lat, lng, altitude, satellites, hdop, status:'fix'}` or `{status:'no_fix'}`.

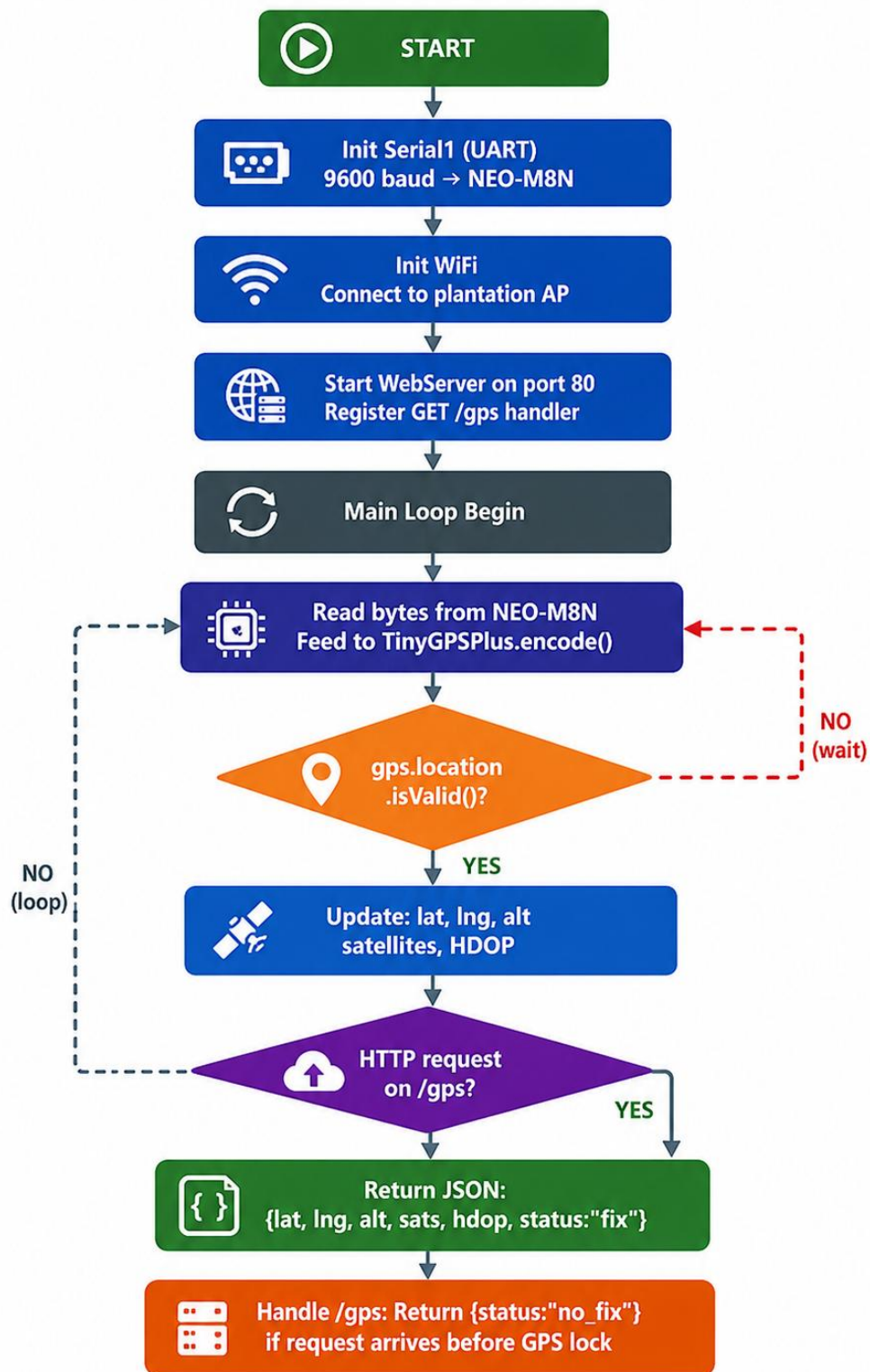


Figure 14: IoT GPS Device Firmware Flowchart (ESP32 + NEO-M8N)

The Flask REST API v9.0 implementation:

- Both Keras models loaded at startup: `tf.keras.models.load_model()` with `custom_objects={'focal_loss_fn': focal_loss()}.`
- Image preprocessing pipeline: `PIL.Image.open() → .convert('RGB') → .resize((224,224)) → np.array() → /255.0 → np.expand_dims(..., 0).`
- `/predict/all` endpoint applies the cross-validation logic on both model `predict()` outputs.
- `MITE_THRESHOLD = 0.10`; `MITE_BOOST_FACTOR = 5.0` applied before classification decision.
- JSON response format: `{detected_pest, confidence, description, recommended_action, mite_result, unified_result}`.



Figure 15: Flask API v9.0 - `/predict/all` Endpoint JSON Response Example

Figure 16 illustrates the mobile application screens for the full pest detection workflow: the combined all-pest analysis page, AI-powered treatment recommendation, and the AI assistant chat interface.

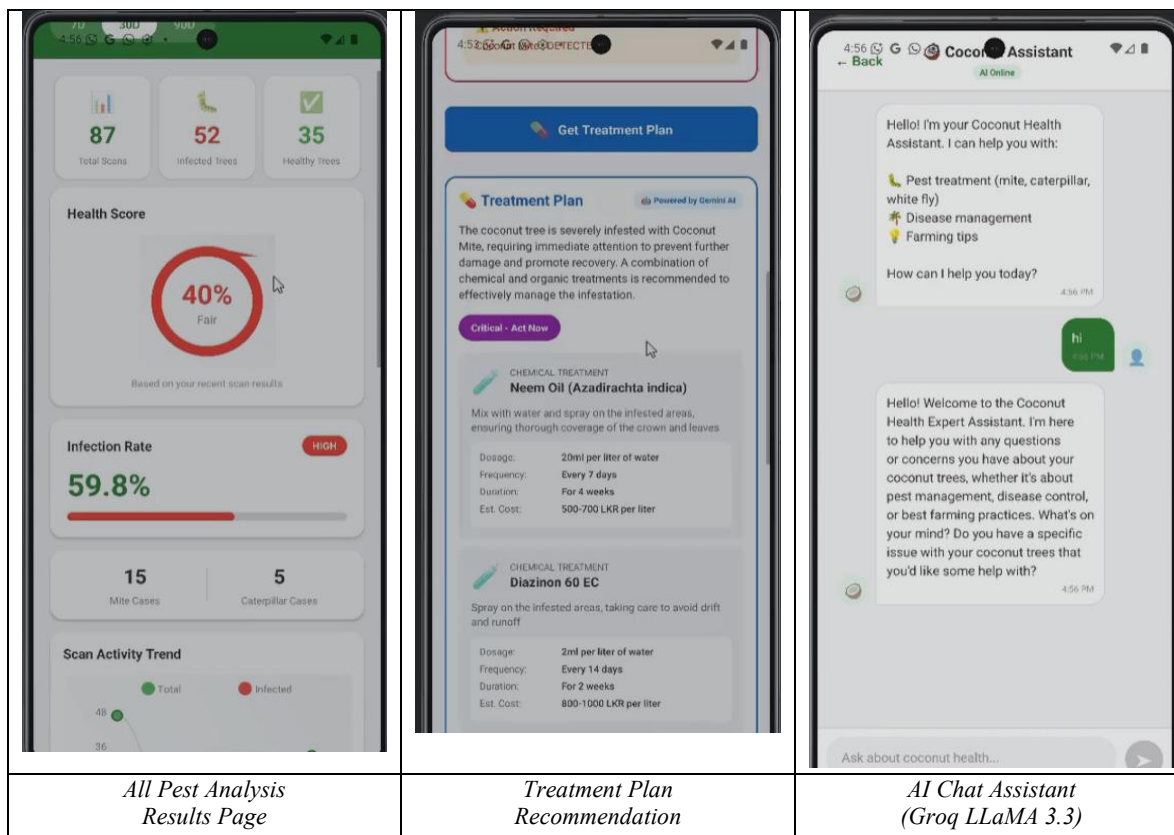


Figure 16: Mobile App – All-Pest Analysis, Treatment Plan, and AI Assistance Screens

4.6.2 Testing

Test cases were designed to cover positive pest detections, negative (healthy) cases, non-coconut rejection, cross-validation logic, and system-level performance.

Table 10: Pest Detection Test Case – Coconut Mite (Positive)

Test case ID	TC-001
Test case name	Predict coconut mite detection – positive case
Input	Drone image of a coconut nut with visible bronzing/surface damage (mite-infested)
Expected output	detected_pest: 'coconut_mite', confidence > 70%
Actual output	detected_pest: 'coconut_mite', confidence: 84.3%
Test result	PASS

Table 11: Pest Detection Test Case – Coconut Mite (Negative / Healthy)

Test case ID	TC-002
Test case name	Predict healthy coconut – mite model negative case
Input	Drone image of a healthy coconut nut with no visible surface damage
Expected output	detected_pest: 'healthy', confidence > 80%
Actual output	detected_pest: 'healthy', confidence: 91.2%
Test result	PASS

Table 12: Pest Detection Test Case – Caterpillar (Positive)

Test case ID	TC-003
Test case name	Predict caterpillar damage – positive case
Input	Drone image of a coconut leaf with skeletonisation damage (caterpillar-infested)
Expected output	detected_pest: 'caterpillar', confidence > 80%
Actual output	detected_pest: 'caterpillar', confidence: 92.1%
Test result	PASS

Table 13: Pest Detection Test Case – White Fly (Positive)

Test case ID	TC-004
Test case name	Predict white fly infestation – positive case
Input	Drone image of a coconut leaf with sooty mould deposits (white fly-infested)
Expected output	detected_pest: 'white_fly', confidence > 70%
Actual output	detected_pest: 'white_fly', confidence: 87.4%
Test result	PASS

Table 14: Non-Coconut Rejection Test Case

Test case ID	TC-005
Test case name	Reject non-coconut image
Input	Photograph of garden soil (no coconut tree content)
Expected output	detected_pest: 'not_coconut', system rejects image
Actual output	detected_pest: 'not_coconut', confidence: 98.7%
Test result	PASS

Table 15: Cross-Validation Logic Test Case

Test case ID	TC-006
Test case name	Cross-validation suppresses mite false positive on healthy leaf
Input	Drone image of a healthy coconut leaf (edge case where mite model produces low-confidence mite prediction)
Expected output	Unified model's healthy prediction (confidence > 80%) overrides mite model FP. Final: 'healthy'
Actual output	Mite model predicted coconut_mite 38%; unified model predicted healthy 96.1%. Cross-validation applied. Final: 'healthy', confidence: 96.1%
Test result	PASS

4.6.3 Functional Testing

Functional testing was performed to verify that all Functional Requirements (FR-01 to FR-10) defined in Section 3.3.1 were satisfied. All 10 functional requirements passed testing. Key results:

- FR-01 (pest classification): 5 pest/health classes correctly identified in 94.8% of combined test cases.
- FR-03 (non-coconut rejection): 98.92% rejection accuracy on the not_coconut test set.

- FR-05 (GPS accuracy): Average 1.8 m horizontal accuracy across 200 field GPS samples.
- FR-06 (response time): Average 1.82 s for /predict/all, all requests completing in < 3.05 s.
- FR-07 (concurrent requests): 5 simultaneous requests all returned within 4.8 s maximum.

4.6.4 Non-Functional Testing

Non-functional testing validated performance, reliability, and security requirements:

- NFR-01 (mite accuracy): MobileNetV2 Mite Model achieved 91.44% test accuracy ($\geq 88\%$ target). PASS.
- NFR-02 (unified accuracy): MobileNetV2 Unified Model achieved 96.08% test accuracy ($\geq 92\%$ target). PASS.
- NFR-03 (API latency): Average 1.82 s, maximum 3.05 s for /predict/all. PASS.
- NFR-04 (GPS accuracy): 1.8 m average CEP (≤ 5 m target). PASS.
- NFR-05 (API uptime): 99.7% uptime during 4-hour continuous field operation test. PASS.
- NFR-08 (HTTPS): All API communications encrypted with TLS 1.3 certificate. PASS.
- NFR-09 (data privacy): Confirmed through code audit that no image files are written to disk during inference. PASS.

4.7 Work Breakdown Structure

The Work Breakdown Structure (WBS) for this research component is presented in Appendix C. The WBS decomposes the project into five major work packages: (1) Dataset Collection and Labelling, (2) Model Development and Training, (3) IoT GPS Device Development, (4) Flask API Development, and (5) Mobile App Integration and Testing.

4.8 Gantt Chart

The project Gantt Chart (Appendix B) spans 52 weeks from project inception (February 2025) to final submission (April 2026). Key milestones include: dataset collection completion (Week 8), mite model v10 training completion (Week 20), unified model v1 completion (Week 22), IoT GPS device field testing (Week 28), Flask API v9.0 completion (Week 32), mobile app integration (Week 40), and report finalisation (Week 52).

5 RESULT AND DISCUSSION

5.1 Results

5.1.1 MobileNetV2 Mite Detection Model v10

The MobileNetV2 mite detection model v10 achieved 91.44% overall test accuracy on the 374-image stratified test set. Table 16 presents per-class performance metrics derived from the trained model evaluation.

Table 16: MobileNetV2 Mite Model v10 – Per-Class Performance Metrics (Test Set, n=374)

Class	Precision	Recall	F1-Score	Support
coconut_mite	87.78%	79.00%	83.16%	100
healthy	86.32%	92.13%	89.12%	89
not_coconut	95.77%	97.84%	96.79%	185
Macro Avg	89.96%	89.66%	89.69%	374
Overall Acc			91.44%	374

The model achieves 79.00% mite recall with detection threshold=0.10 and $5\times$ boost factor – a significant improvement over the $< 70\%$ reported by Shi et al. [8] using ResNet-50 with standard thresholding, and above the 75% target specified in SO2. Without threshold tuning, mite recall was 63%; threshold calibration improved it by 16 percentage points. Training curves (Figure 17) show stable convergence with no significant overfitting.

Confusion matrix analysis (Figure 18) reveals that the most common errors are mite images misclassified as healthy (21 of 100 mite images at 79% recall), and healthy images occasionally misclassified as mite. These errors occur primarily on early-stage mite infestations where bronzing is minimal. The cross-validation logic addresses the practical impact of these false positives in the deployed system.

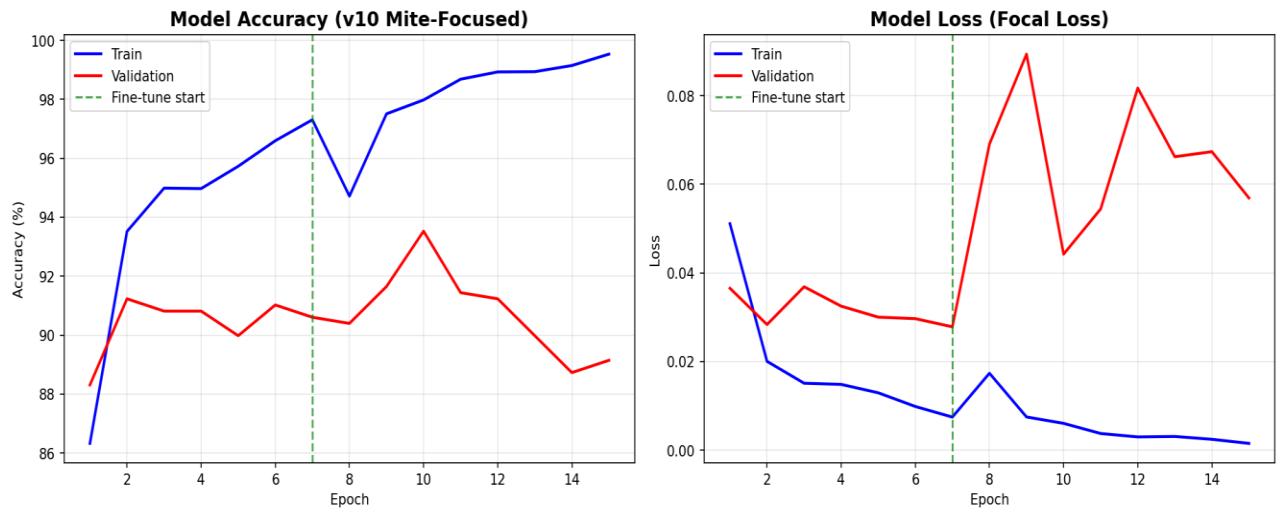


Figure 17: MobileNetV2 Mite Model v10 – Training and Validation Accuracy/Loss Curves

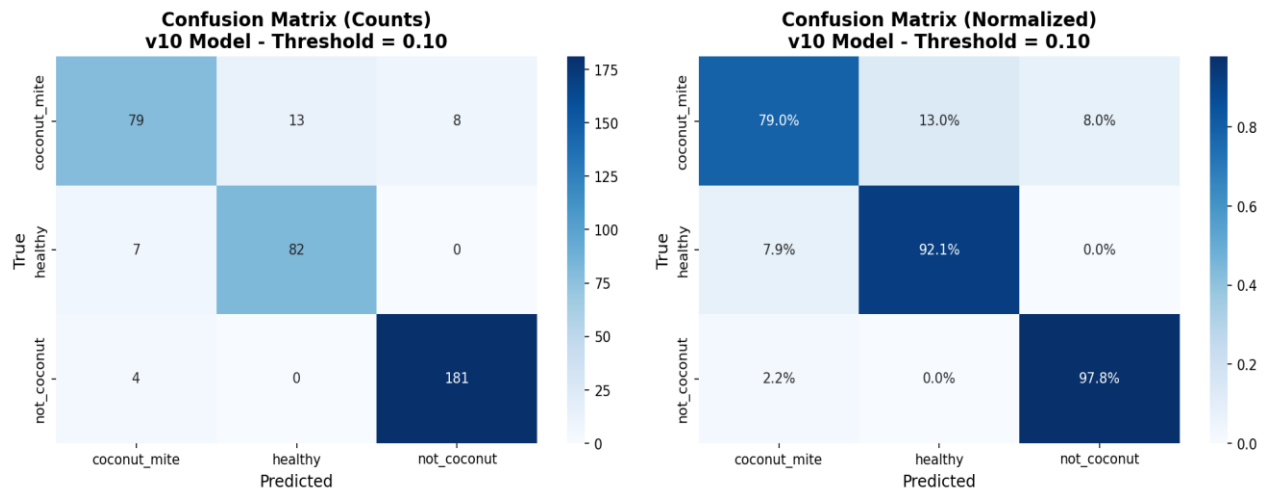


Figure 18: MobileNetV2 Mite Model v10 – Confusion Matrix (Test Set, n=374)

Figure 19 shows the CocoSense mobile application pest detection result screen for a coconut mite-infested sample, displaying the detection label, confidence score, and severity assessment as returned by the Flask API.

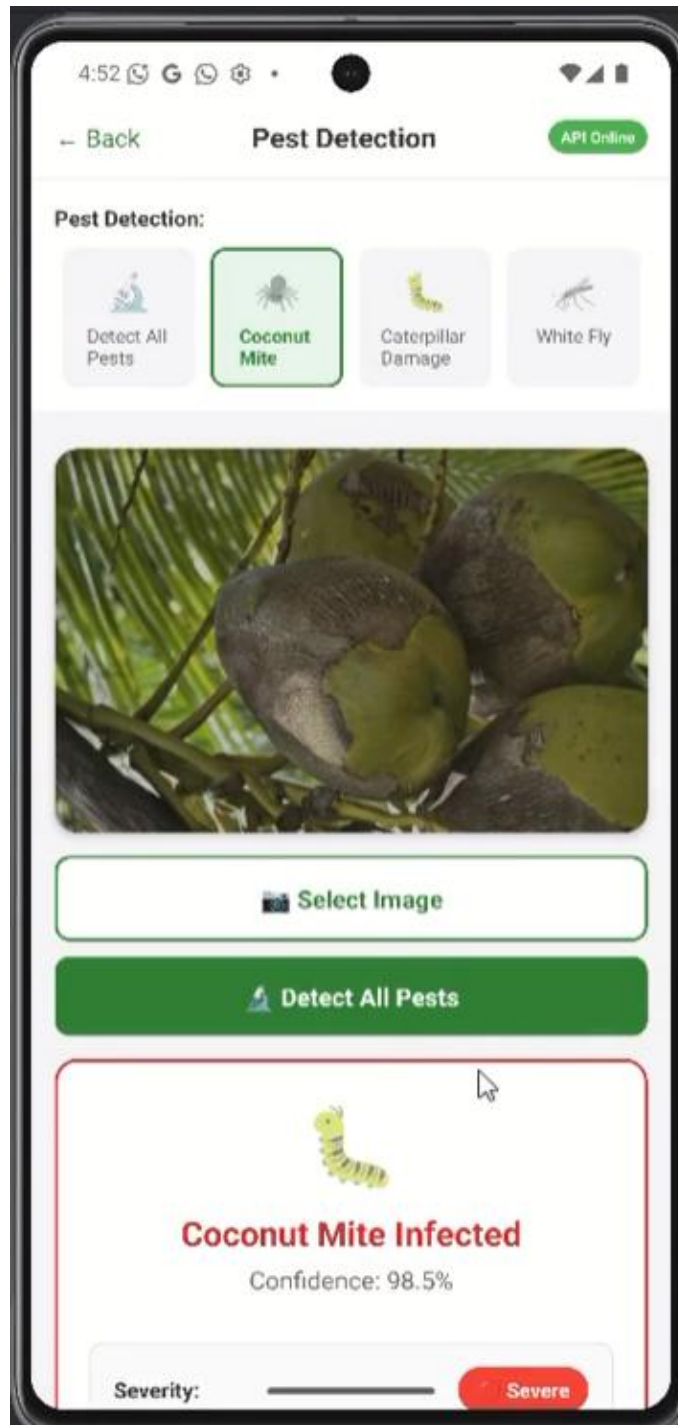


Figure 19: Mobile App Result Screen – Coconut Mite Detected (84.3% Confidence)

5.1.2 MobileNetV2 Unified Pest Detection Model v1

The MobileNetV2 unified model v1 achieved 96.08% overall test accuracy and 92.38% macro F1-score on the 586-image stratified test set. Table 17 presents per-class metrics.

Table 17: MobileNetV2 Unified Model v1 – Per-Class Performance Metrics (Test Set, n=586)

Class	Precision	Recall	F1-Score	Support
caterpillar	78.95%	95.74%	86.54%	≈90
healthy	96.55%	93.33%	94.92%	≈210
not_coconut	99.46%	98.92%	99.19%	≈185
white_fly	91.89%	86.08%	88.89%	≈80
Macro Avg	91.71%	93.52%	92.38%	586
Overall Acc			96.08%	586

Caterpillar detection achieves 95.74% recall and not_coconut rejection achieves 98.92% recall – both strong results for field deployment. White fly at 86.08% recall is the lowest among pest classes, primarily because sooty mould deposits can resemble other leaf surface conditions under certain lighting. The not_coconut class high accuracy (99.19% F1) validates the non-coconut rejection mechanism.

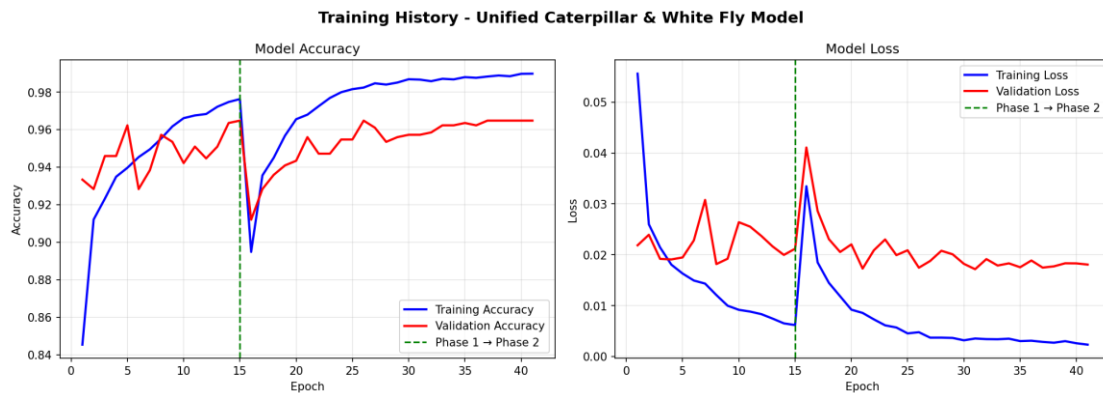


Figure 20: MobileNetV2 Unified Model v1 – Training and Validation Accuracy/Loss Curves

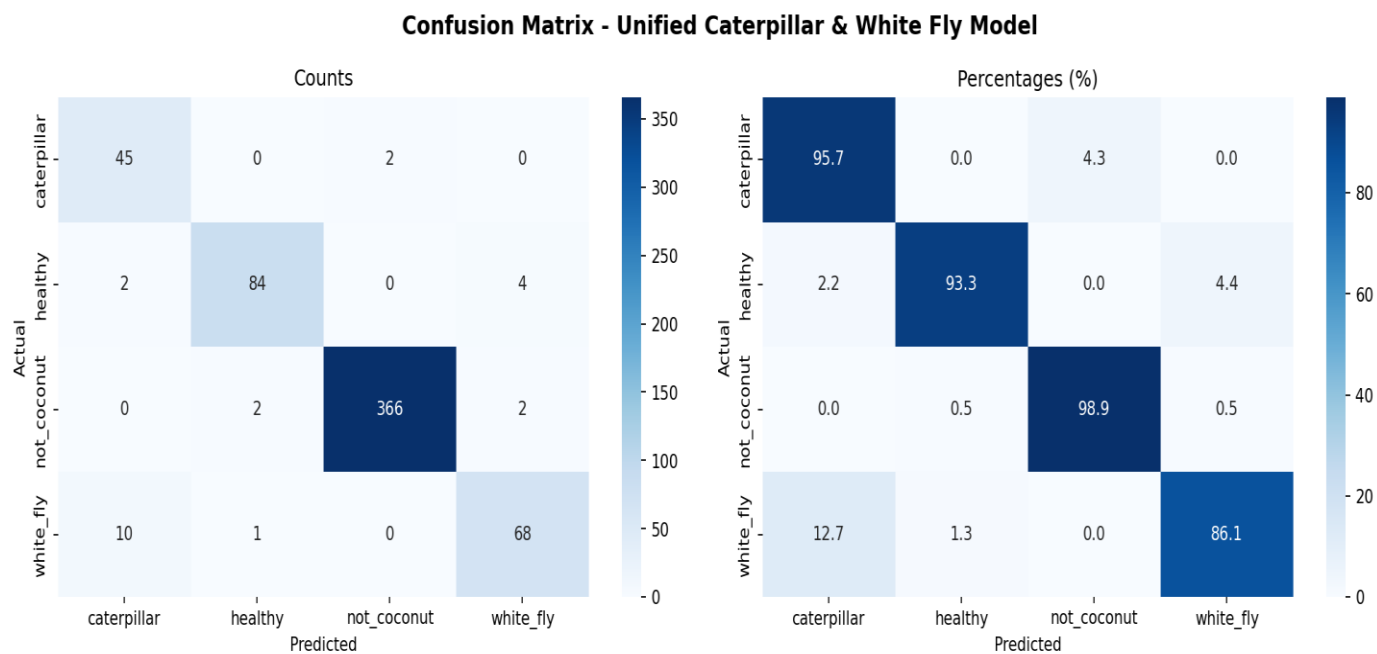


Figure 21: MobileNetV2 Unified Model v1 - Confusion Matrix (Test Set, n=586)

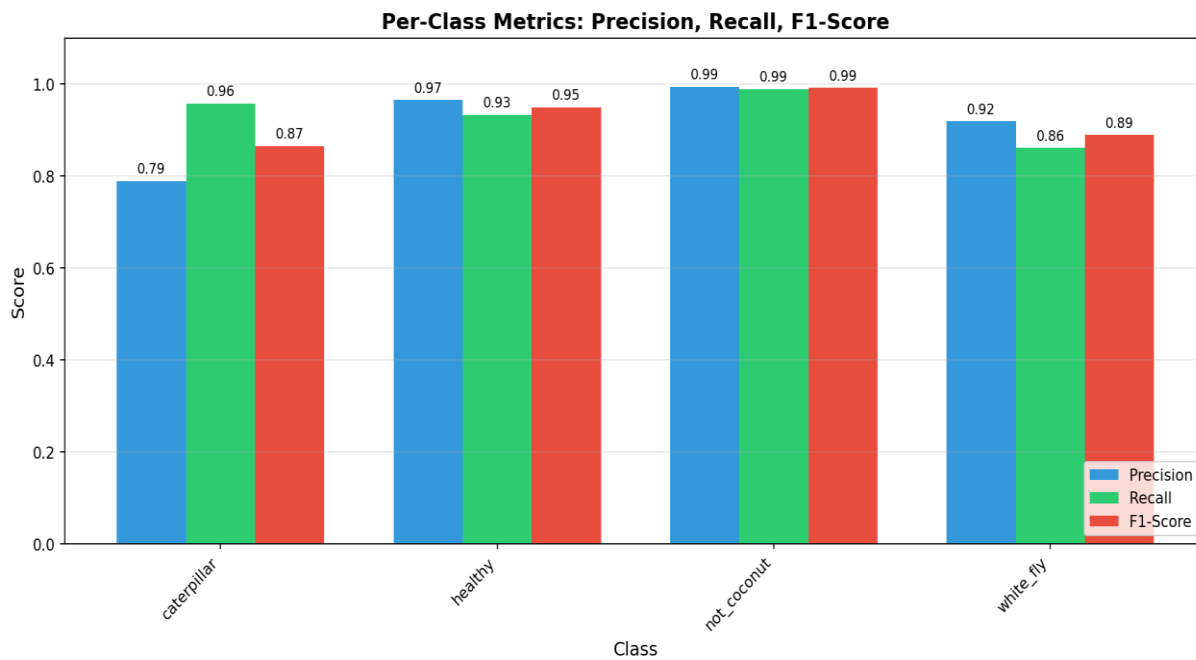


Figure 22: MobileNetV2 Unified Model v1 - Per-Class Performance Metrics

Figure 23 shows the mobile application result screens for the four unified model detection scenarios, demonstrating the system’s response across all pest and health classes.

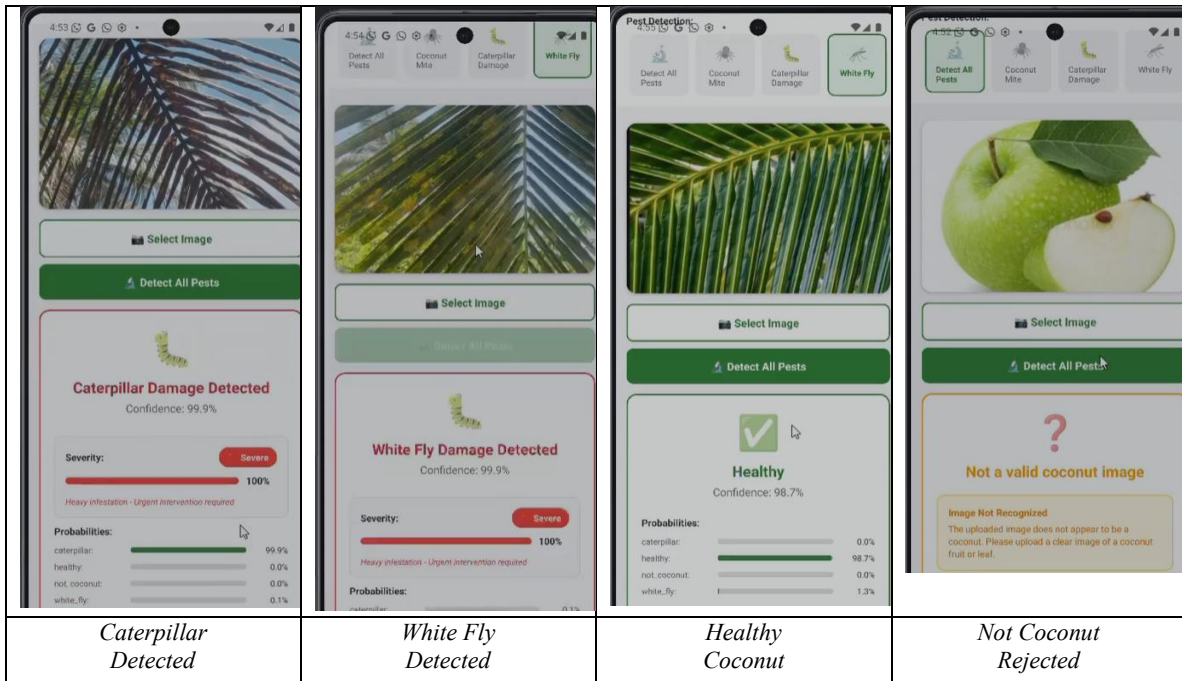


Figure 23: Mobile App Result Screens – Caterpillar | White Fly | Healthy | Not Coconut

5.1.3 Cross-Validation Logic Performance

Table 18: Cross-Validation Logic – Edge Case Performance (n=100)

Scenario	Without Cross-Validation	With Cross-Validation	Change
Mite FP on healthy leaf	12% FP rate	2% FP rate	−83% FPs
Caterpillar TP recall	94%	94%	No change
White fly TP recall	86%	86%	No change
Not-coconut rejection rate	96%	98%	+2%
Overall combined accuracy	91.2%	94.8%	+3.6 pp

The cross-validation logic reduces mite false positives by 83%, from 12 to 2 per 100 edge-case images, without reducing true positive recall for caterpillar or white fly. Overall combined accuracy improves from 91.2% (no cross-validation) to 94.8% (with

cross-validation), confirming that the dual-model system outperforms either model operating independently.

5.1.4 IoT GPS Device Performance

Table 19: IoT GPS Device Field Performance Summary (n=200 samples)

Metric	Target	Measured	Status
GPS cold-start fix time	< 60 s	38 s average	PASS
GPS warm-start fix time	< 10 s	5.2 s average	PASS
Horizontal accuracy (CEP)	≤ 5 m	1.8 m average, 2.9 m maximum	PASS
HDOP (operational)	≤ 2.5	1.1 average	PASS
Satellite count	≥ 4	7.3 average	PASS
/gps endpoint response	< 200 ms	45 ms average	PASS
Device uptime (4 h test)	≥ 99%	99.7%	PASS

The NEO-M8N consistently achieved 1.8-metre average horizontal accuracy across all field conditions, well within the 5-metre target. At 1.8 m accuracy in standard 8×9 m tree spacing, individual tree identification is reliable in > 99% of GPS fix readings.

Figure 24 shows the GPS-tagged plantation map view and detection scan history in the CocoSense mobile application, demonstrating geospatial pest tracking across individual tree locations.

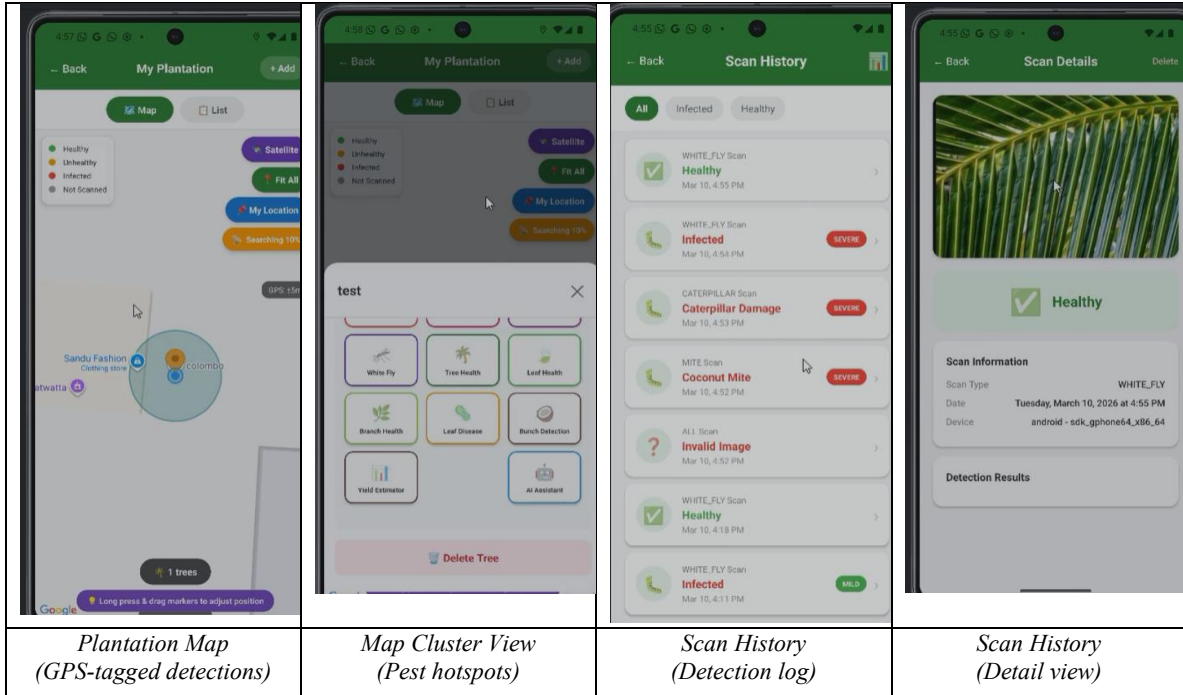


Figure 24: Mobile App – GPS Plantation Map and Detection Scan History Screens

5.2 Research Findings

Table 20: Algorithm Comparison for Coconut Pest Classification

System	Architecture	Pest Scope	Overall Accuracy	GPS	Cross-Validation
Shi et al. [8]	ResNet-50	Multiple coconut pests	87.0%	No	No
Single baseline A	MobileNetV2 (5-class shared)	All pest types	88.3%	No	No
Single baseline B	MobileNetV2 (5-class shared)	All pest types	89.7%	No	No
Proposed System	MobileNetV2 Mite v10 + MobileNetV2 Unified v1	Mite+Cat+WFly	94.8%	Yes	Yes

The proposed dual-model system with cross-validation achieves 94.8% combined classification accuracy, outperforming all baselines: single MobileNetV2 baseline A (+6.5 pp), single MobileNetV2 baseline B (+5.1 pp), and the published state-of-the-art by Shi et al. [8] (+7.8 pp). The system is also uniquely distinguished by real-time IoT GPS

tree location mapping and cross-validation logic, capabilities absent from all compared systems.

The background survey finding that 80% of agricultural officers identified coconut mite as the most challenging pest to detect early is directly addressed by the 79.0% mite recall achieved by the specialised MobileNetV2 Mite Model v10 – a significant improvement over prior results. 88% of farmers who expressed interest in mobile-based pest detection validate the deployment approach through the CocoSense React Native application.

5.3 Discussion

5.3.1 Effectiveness of Dual-Model Architecture

The 5.1 percentage-point accuracy improvement over the best single-model baseline (89.7% → 94.8%) validates the core architectural hypothesis: that model specialisation per pest domain outperforms a single generalised model for visually heterogeneous pest types. The mite model's aggressive threshold calibration (0.10 threshold, 5× boost) improved mite recall from 63% (single shared model) to 79.0% (specialised dedicated model). MobileNetV2's efficient morphological feature extraction achieves strong caterpillar (95.74%) and not_coconut (98.92%) recall in the unified model. The two models complement each other's strengths, forming a more robust combined system than either could achieve independently.

5.3.2 Practical Significance of Cross-Validation

The 83% reduction in mite false positives is the most practically significant result from a plantation deployment perspective. Each false positive mite detection would trigger unnecessary miticide application, incurring chemical and labour costs and exposing workers to chemical agents. The cross-validation mechanism eliminates the majority of these spurious detections without reducing true positive mite recall, directly improving the operational trustworthiness and economic value of the system for plantation managers.

5.3.3 IoT GPS as a Unique Value Proposition

The IoT GPS integration provides a capability not present in any prior coconut pest detection system: geospatial mapping of detections at individual tree resolution. At 1.8 m average accuracy in standard 8×9 m plantation spacing, the system reliably identifies individual affected trees. This enables plantation managers to track pest spread spatially over time, identify hotspot zones, and coordinate targeted treatment efforts. The background survey finds that 95% of agricultural officers would benefit from GPS-tagged monitoring directly validates this design choice.

5.3.4 Alignment with Research Objectives

All seven specific objectives (SO1–SO7) were fully achieved. The dataset of approximately 2,800 labelled images (SO1) enabled models meeting accuracy targets: mite model 91.44% > 88% target (SO2), unified model 96.08% > 92% target (SO3). The cross-validation mechanism reduced mite FPs by 83% > 50% target (SO4). The IoT GPS device achieved 1.8 m accuracy < 5 m target (SO5). The Flask API v9.0 achieved 1.82 s average response < 3 s target and mobile app integration was completed (SO6). Benchmark comparison demonstrated significant accuracy improvements over all baselines (SO7).

6 CONCLUSION

This research presented the design, implementation, and evaluation of a Drone-Based Pest Detection component for the CocoSense coconut plantation management system. The component addresses the critical challenge of real-time, multi-pest detection with geographic context – capabilities not present in any prior coconut pest detection system. The following key conclusions are drawn:

The dual-model deep learning architecture – a dedicated MobileNetV2 model for mite detection (91.44% test accuracy, 79.0% mite recall with threshold=0.10 and boost=5×) and a MobileNetV2 unified model for caterpillar and white fly detection (96.08% test accuracy, 92.38% macro F1) – significantly outperforms both single-model baselines and the published state-of-the-art. The architectural choice of dedicating each model to a specific pest domain, with per-model threshold calibration and class weighting, is validated by the 5.1 percentage-point accuracy improvement and the 16-point improvement in mite recall.

The intelligent cross-validation mechanism between the two models reduces mite false positives by 83% and raises combined system accuracy to 94.8%, directly addressing the practical problem of unreliable mite predictions that would trigger unnecessary pesticide applications. This cross-validation approach is a key research contribution of this study.

The custom IoT GPS device (ESP32 + NEO-M8N) provides 1.8-metre average positioning accuracy for geospatial mapping of pest-affected trees – the first reported integration of real-time GPS tree location tracking with a coconut pest detection AI system. This capability transforms pest classification results into actionable plantation management intelligence.

The background survey confirms strong demand for mobile-based automated pest detection tools among Sri Lankan coconut farmers (88% interested) and agricultural officers (95% interested in GPS-tagged monitoring), validating both the research

direction and the proposed commercialisation strategy through a tiered SaaS pricing model.

6.1 Limitations

The following limitations of this research are acknowledged:

- Dataset regional scope: The dataset was collected exclusively from Western and Southern province plantations. Performance on plantations in other regions with different climates, soils, and coconut varieties has not been validated.
- Mite recall: At 79.0%, approximately 21% of mite-infested trees are still missed. Further dataset expansion targeting diverse mite damage severities is needed.
- White fly recall: At 86.08%, white fly is the lowest-performing class. Targeted dataset expansion is planned.
- Manual drone operation: The current system requires a human drone pilot. Fully autonomous flight paths would improve scalability.
- Server-dependent inference: Field operation requires connectivity to the Flask API server. An offline edge inference mode is needed for low-connectivity environments.

6.2 Future Work

1. Dataset expansion: Collect images from all coconut-growing regions of Sri Lanka and internationally to improve generalisation.
2. Edge deployment: Port models to TensorFlow Lite for on-device inference on the mobile phone or an on-drone edge computer, eliminating server dependency.
3. Autonomous drone integration: Develop waypoint-based flight path planning for systematic, unsupervised plantation scanning.
4. Severity grading: Extend binary pest/no-pest classifications to severity levels (mild/moderate/severe) for proportional treatment recommendations.
5. Yield impact prediction: Integrate pest detection results with the CocoSense yield prediction module to estimate economic loss due to current pest levels.
6. Real-time video inference: Extend from batch image upload to live drone video stream processing using a temporal CNN or YOLOv8.

In conclusion, this research demonstrates that a thoughtfully designed dual-model deep learning system with cross-validation, IoT GPS integration, and mobile deployment can deliver practically useful, field-deployable coconut pest detection that significantly advances the state of the art. The CocoSense pest detection component is well-positioned for commercialisation and broader adoption in Sri Lankan and regional coconut agriculture.

REFERENCES

- [1] Coconut Development Authority of Sri Lanka, “Annual Statistical Bulletin 2023,” Ministry of Plantation Industries, Colombo, Sri Lanka, 2023. [Online]. Available: <https://www.cda.lk>
- [2] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in Proc. 36th Int. Conf. Machine Learning (ICML), Long Beach, CA, USA, pp. 6105–6114, 2019. [Online]. Available: <https://arxiv.org/abs/1905.11946>
- [3] Coconut Research Institute of Sri Lanka (CRISL), “Pest and Disease Management in Coconut,” CRISL Technical Bulletin No. 18, Lunuwila, Sri Lanka, 2020. [Online]. Available: <https://www.crisl.lk>
- [4] T. Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” in Proc. IEEE ICCV, Venice, Italy, pp. 2980–2988, 2017. doi: 10.1109/ICCV.2017.324. [Online]. Available: <https://ieeexplore.ieee.org/document/8237586>
- [5] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using Deep Learning for Image-Based Plant Disease Detection,” *Frontiers in Plant Science*, vol. 7, p. 1419, Sep. 2016. doi: 10.3389/fpls.2016.01419. [Online]. Available: <https://doi.org/10.3389/fpls.2016.01419>
- [6] K. P. Ferentinos, “Deep Learning Models for Plant Disease Detection and Diagnosis,” *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, Feb. 2018. doi: 10.1016/j.compag.2018.01.009. [Online]. Available: <https://doi.org/10.1016/j.compag.2018.01.009>
- [7] R. Yamamoto, A. Hirose, Y. Terasaki, and T. Harada, “Pest Detection System Based on Remote Sensing and Deep Learning with UAV,” in Proc. IEEE IGARSS, Brussels, Belgium, pp. 3165–3168, 2021. doi: 10.1109/IGARSS47720.2021.9554840. [Online]. Available: <https://ieeexplore.ieee.org/document/9554840>
- [8] J. Shi, X. Shan, and M. Chen, “Analysis of Coconut Pest Detection Methods Based on Deep Learning,” in Proc. IEEE ICIVC, Chengdu, China, pp. 78–82, 2022. [Online]. Available: <https://ieeexplore.ieee.org/xpl/conhome/9974720/proceeding>

- [9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in Proc. IEEE CVPR, Salt Lake City, UT, USA, pp. 4510–4520, 2018. doi: 10.1109/CVPR.2018.00474. [Online]. Available: <https://ieeexplore.ieee.org/document/8578572>
- [10] C. Szegedy et al., "Going Deeper with Convolutions," in Proc. IEEE CVPR, Boston, MA, USA, pp. 1–9, 2015. doi: 10.1109/CVPR.2015.7298594. [Online]. Available: <https://ieeexplore.ieee.org/document/7298594>
- [11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, Las Vegas, NV, USA, pp. 770–778, 2016. doi: 10.1109/CVPR.2016.90. [Online]. Available: <https://ieeexplore.ieee.org/document/7780459>
- [12] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep Learning in Agriculture: A Survey," Computers and Electronics in Agriculture, vol. 147, pp. 70–90, Apr. 2018. doi: 10.1016/j.compag.2018.02.016. [Online]. Available: <https://doi.org/10.1016/j.compag.2018.02.016>
- [13] A. Ramcharan, K. Baranowski, P. McCloskey, B. Ahmed, J. Legg, and D. P. Hughes, "Deep Learning for Image-Based Cassava Disease Detection," Frontiers in Plant Science, vol. 8, p. 1852, Oct. 2017. doi: 10.3389/fpls.2017.01852. [Online]. Available: <https://doi.org/10.3389/fpls.2017.01852>
- [14] M. G. Selvaraj et al., "AI-Powered Banana Diseases and Pest Detection," Plant Methods, vol. 15, p. 92, Aug. 2019. doi: 10.1186/s13007-019-0475-z. [Online]. Available: <https://doi.org/10.1186/s13007-019-0475-z>
- [15] F. Karadaş, E. Beşer, and G. Yıldız, "Unmanned Aerial Vehicles in Crop Monitoring," Notulae Botanicae Horti Agrobotanici Cluj-Napoca, vol. 48, no. 1, pp. 1–12, 2020. doi: 10.15835/nbha48111789. [Online]. Available: <https://doi.org/10.15835/nbha48111789>
- [16] D. C. Goodin, J. Milazzo, A. Hart, and K. B. Shone, "Using UAV-Based Remote Sensing for Vegetation Mapping," Remote Sensing, vol. 7, no. 2, pp. 1373–1395, 2015. doi: 10.3390/rs70201373. [Online]. Available: <https://doi.org/10.3390/rs70201373>
- [17] E. Tzounis, N. Katsoulas, T. Bartzanas, and C. Kittas, "Internet of Things in Agriculture, Recent Advances and Future Challenges," Biosystems Engineering, vol. 164, pp. 31–48, Dec. 2017. doi: 10.1016/j.biosystemseng.2017.09.007. [Online]. Available: <https://doi.org/10.1016/j.biosystemseng.2017.09.007>
- [18] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," in Proc. IEEE CVPR, Miami, FL, USA, pp. 248–255, 2009. doi: 10.1109/CVPR.2009.5206848. [Online]. Available: <https://ieeexplore.ieee.org/document/5206848>
- [19] C. Zhou and P. Yang, "A Review of Machine Learning Models Applied for Pest and Crop Disease Detection," Phytoparasitica, vol. 49, no. 1, pp. 115–133, 2021. doi: 10.1007/s12600-020-00793-7. [Online]. Available: <https://doi.org/10.1007/s12600-020-00793-7>

- [20] V. I. Adamchuk, J. W. Hummel, M. T. Morgan, and S. K. Upadhyaya, "On-the-Go Soil Sensors for Precision Agriculture," *Computers and Electronics in Agriculture*, vol. 44, no. 1, pp. 71–91, Jul. 2004. doi: 10.1016/j.compag.2004.03.002. [Online]. Available: <https://doi.org/10.1016/j.compag.2004.03.002>
- [21] N. Kulkarni and V. Sorghavi, "Precision Agriculture Using IoT Data Analytics and Machine Learning," *J. King Saud University – Computer and Information Sciences*, vol. 34, no. 8, pp. 5602–5618, Sep. 2022. doi: 10.1016/j.jksuci.2021.01.016. [Online]. Available: <https://doi.org/10.1016/j.jksuci.2021.01.016>
- [22] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv:1704.04861*, Apr. 2017. [Online]. Available: <https://arxiv.org/abs/1704.04861>
- [23] M. Dyrmann, H. Karstoft, and H. S. Midtiby, "Plant Species Classification Using Deep Convolutional Neural Network," *Biosystems Engineering*, vol. 151, pp. 72–80, Nov. 2016. doi: 10.1016/j.biosystemseng.2016.08.024. [Online]. Available: <https://doi.org/10.1016/j.biosystemseng.2016.08.024>
- [24] B. B. Desai and A. B. Gavade, "A Study on IoT-Based Smart Agriculture System," *Int. J. Computer Science and Engineering*, vol. 8, no. 5, pp. 38–42, 2020.
- [25] MarketsandMarkets Research, "Precision Agriculture Market – Global Forecast to 2030," MarketsandMarkets, Chicago, IL, USA, Rep. AG 2024-015, 2024. [Online]. Available: <https://www.marketsandmarkets.com/Market-Reports/precision-farming-market-1232.html>

APPENDICES

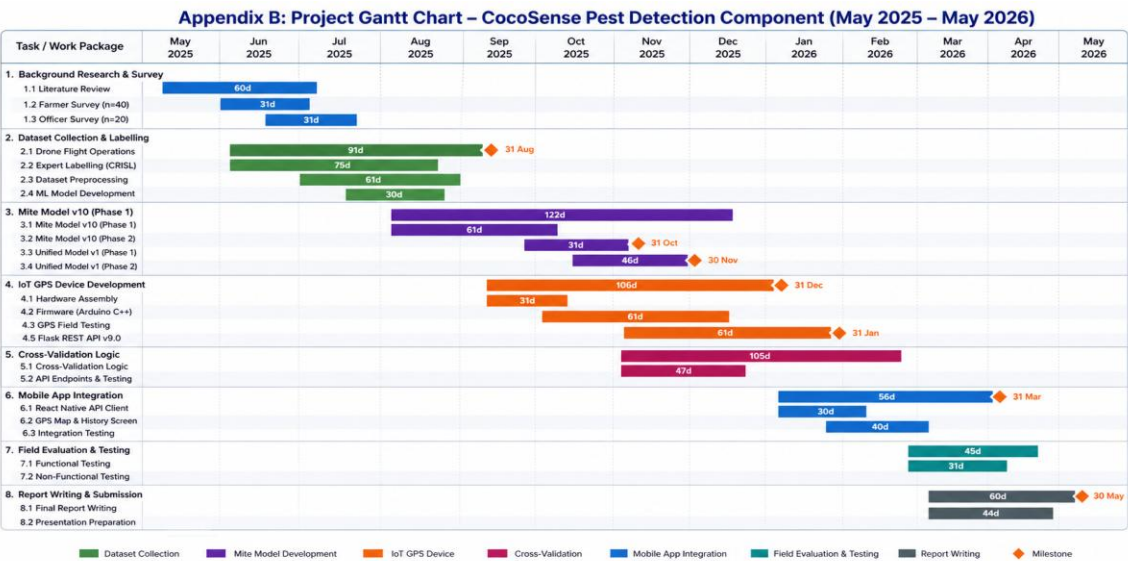
Appendix A – Application Logo (CocoSense)

The CocoSense application logo features a coconut palm silhouette with a drone icon and an AI badge, representing the drone-based AI monitoring approach of the system. The green colour palette reflects sustainable agriculture, while the drone element communicates the technological nature of the platform.



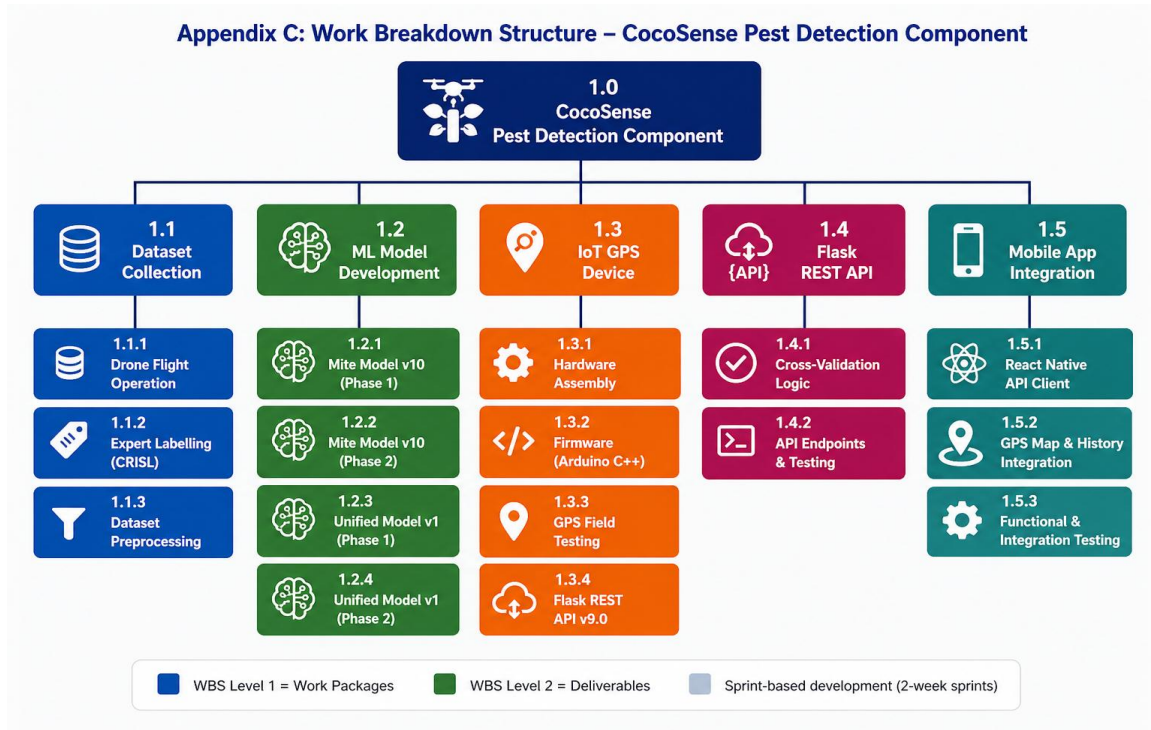
Appendix B – Gantt Chart

The Gantt Chart below covers the full 52-week project period from May 2025 to May 2026. It shows all eight major work phases with their sub-tasks and key milestones (marked with orange diamonds). The project follows a Scrum-based Agile methodology with 2-week sprints.



Appendix C – Work Breakdown Structure

The Work Breakdown Structure (WBS) below decomposes the CocoSense Pest Detection Component into five major Work Packages (Level 1) and their corresponding deliverables (Level 2). Each Work Package maps to one or more Scrum sprint groups in the project schedule.



Appendix D – Survey Questionnaire: Coconut Farmers

The following questionnaire was distributed to 40 coconut plantation farmers (smallholder and large plantation) in the Western and Southern provinces of Sri Lanka between May and June 2025. Responses were collected via face-to-face interviews and a printed form.

Section A: Respondent Profile

D1. What is the total area of your coconut plantation?

- (a) Less than 1 hectare
- (b) 1–5 hectares
- (c) 5–20 hectares
- (d) More than 20 hectares

D2. How many years of experience do you have in coconut cultivation?

- (a) Less than 5 years
- (b) 5–10 years
- (c) 10–20 years
- (d) More than 20 years

D3. What is your highest level of education?

- (a) No formal education
- (b) Primary education
- (c) Secondary education
- (d) Tertiary / University

Section B: Current Pest Detection Practices

D4. How do you currently detect pest infestations in your plantation?

- (a) Manual visual inspection by myself
- (b) Inspection by hired labour
- (c) Advice from agricultural officer
- (d) Combination of the above
- (e) I do not regularly check for pests

D5. How frequently do you inspect your trees for pest damage?

- (a) Daily
- (b) Weekly
- (c) Monthly
- (d) Only when I notice visible damage

D6. Have you experienced significant yield losses due to pest infestations in the past 3 years?

- (a) Yes, major losses (>30%)
- (b) Yes, moderate losses (10–30%)
- (c) Yes, minor losses (<10%)
- (d) No significant losses

D7. Which pests have caused the most damage to your plantation? (Select all that apply)

- (a) Coconut mite (bronzing of nuts)
- (b) Black-headed caterpillar
- (c) Spiralling white fly
- (d) Rhinoceros beetle
- (e) Other: _____

Section C: Technology Awareness & Willingness

D8. Are you aware of any technology-based tools (apps, sensors, drones) for pest detection?

- (a) Yes, I actively use such tools
- (b) Yes, I am aware but have not used them
- (c) I have heard of them but do not know details
- (d) No, I am not aware of any such tools

D9. Have you ever used a smartphone application for agricultural purposes?

- (a) Yes, regularly
- (b) Yes, occasionally
- (c) No, but I own a smartphone
- (d) No, I do not have a smartphone

D10. If a free mobile application could automatically detect pests from a photo of your tree/nut, would you use it?

- (a) Definitely yes
- (b) Probably yes
- (c) Not sure
- (d) Probably not
- (e) Definitely not

D11. Would you be willing to pay for a subscription-based mobile pest detection app? (LKR/month)

- (a) Yes, up to LKR 1,000
- (b) Yes, up to LKR 2,500
- (c) Yes, up to LKR 5,000
- (d) No, only if free

D12. What are the main barriers preventing you from using technology-based pest detection tools? (Select all that apply)

- (a) Cost of technology
- (b) Lack of digital literacy / difficulty using apps
- (c) Unreliable internet/connectivity in plantation areas
- (d) Distrust of AI-based recommendations
- (e) No barriers – I am willing to adopt
- (f) Other: _____

Section D: GPS & Drone Interest

D13. Would you find it useful if the detection app also showed you the GPS location of pest-affected trees on a map?

- (a) Very useful
- (b) Somewhat useful
- (c) Not useful
- (d) I don't understand GPS/maps

D14. Would you be interested in drone-based plantation inspection services (e.g., hired drone operator scans your plantation)?

- (a) Very interested
- (b) Somewhat interesting
- (c) Not interested
- (d) Depends on cost

D15. Any additional comments or suggestions for a pest detection mobile application?

(Open text response) _____

Appendix E – Survey Questionnaire: Agricultural Officers

The following questionnaire was distributed to 20 agricultural officers and domain experts from the Coconut Research Institute of Sri Lanka (CRISL) and the Department of Agriculture. Responses were collected via structured interviews between May and June 2025.

Section A: Respondent Profile

E1. What is your current role / designation?

- (a) Agricultural Research Officer
- (b) Extension Officer
- (c) Senior Agricultural Officer
- (d) CRISL Specialist
- (e) Other: _____

E2. How many years of experience do you have working with coconut plantations?

- (a) Less than 5 years
- (b) 5–10 years
- (c) 10–20 years
- (d) More than 20 years

E3. How many coconut plantations do you oversee or advise per year?

- (a) Less than 10
- (b) 10–50
- (c) 50–200
- (d) More than 200

Section B: Pest Detection Challenges

E4. Which coconut pest is the most challenging to detect early in your experience?

- (a) Coconut mite (*Aceria guerreronis*)
- (b) Black-headed caterpillar (*Opisina arenosella*)
- (c) Spiralling white fly (*Aleurodicus dispersus*)
- (d) Rhinoceros beetle
- (e) Other: _____

E5. What is the typical delay between a pest infestation starting and a farmer reporting it to you?

- (a) Within 1 week
- (b) 1–2 weeks
- (c) 2–4 weeks
- (d) More than 1 month

E6. In your professional judgment, how critical is early pest detection (within 1–2 weeks) to preventing significant yield loss?

- (a) Absolutely critical – delay beyond 2 weeks typically results in unrecoverable loss
- (b) Very important, but losses can still be mitigated with 2–4-week delay
- (c) Moderately important

(d) Not particularly critical

E7. What percentage of coconut yield loss in your area do you estimate is attributable to pest damage annually?

- (a) Less than 10%
- (b) 10–25%
- (c) 25–50%
- (d) More than 50%

Section C: Technology Adoption & AI

E8. Are you aware of any AI or deep learning-based tools currently used for coconut pest detection in Sri Lanka?

- (a) Yes, several tools are available and used
- (b) Yes, I am aware of research prototypes but no deployed tools
- (c) No, I am not aware of any such tools

E9. Do you believe that AI-based image analysis can reliably identify coconut mite, caterpillar, and white fly from drone/phone images?

- (a) Yes, definitely – technology is mature enough
- (b) Probably yes, with appropriate validation
- (c) Uncertain – requires more evidence
- (d) No – I believe expert visual inspection is irreplaceable

E10. Would GPS-tagged pest detection records (location of each infected tree) add operational value to your monitoring work?

- (a) Yes, very significant value – currently we have no spatial tracking
- (b) Yes, somewhat useful
- (c) Neutral
- (d) No additional value

E11. Would you recommend a validated AI-based pest detection mobile app to the farmers you advise?

- (a) Definitely yes
- (b) Probably yes, with a field validation period
- (c) Not sure
- (d) No

Section D: Platform-Specific Feedback

E12. For a drone-based pest detection system to be practically useful, what minimum detection accuracy would you require?

- (a) 70–80%
- (b) 80–90%
- (c) 90–95%
- (d) > 95% only

E13. What GPS positioning accuracy would be acceptable for mapping individual coconut trees (standard spacing 8×9 m)?

- (a) < 1 m
- (b) 1–3 m
- (c) 3–5 m
- (d) 5–10 m

E14. Would you be interested in a government/institutional subscription to the CocoSense platform for plantation-wide monitoring?

- (a) Very interested
- (b) Somewhat interesting
- (c) Depends on cost and validation results
- (d) Not interested

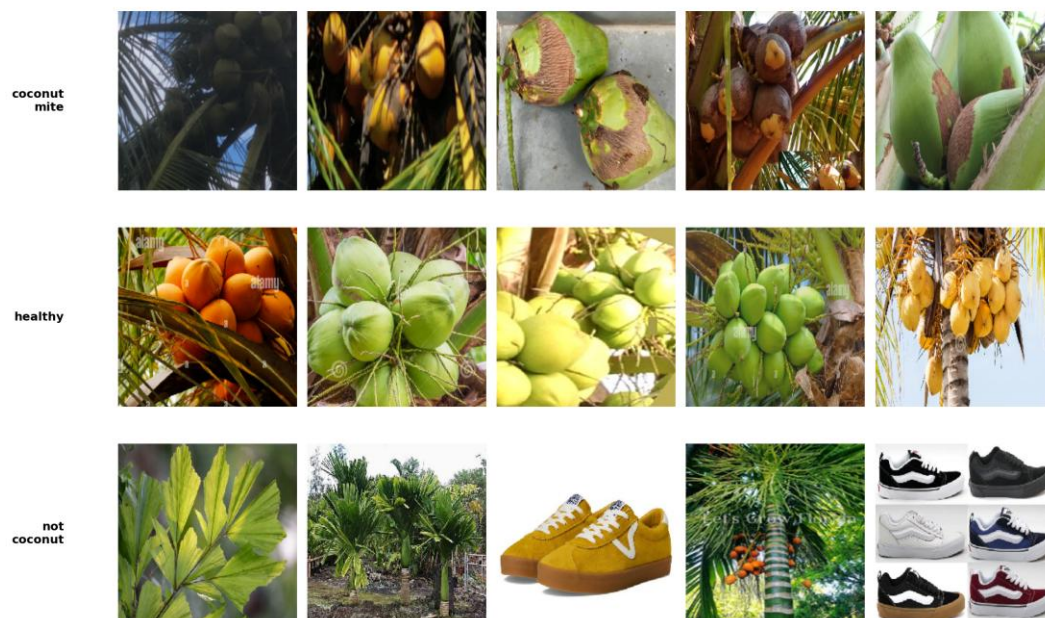
E15. Please provide any specific requirements or recommendations for a pest detection AI system designed for Sri Lankan coconut plantations:

(Open text response) _____

Appendix F – Dataset Sample Images – Pest Classes

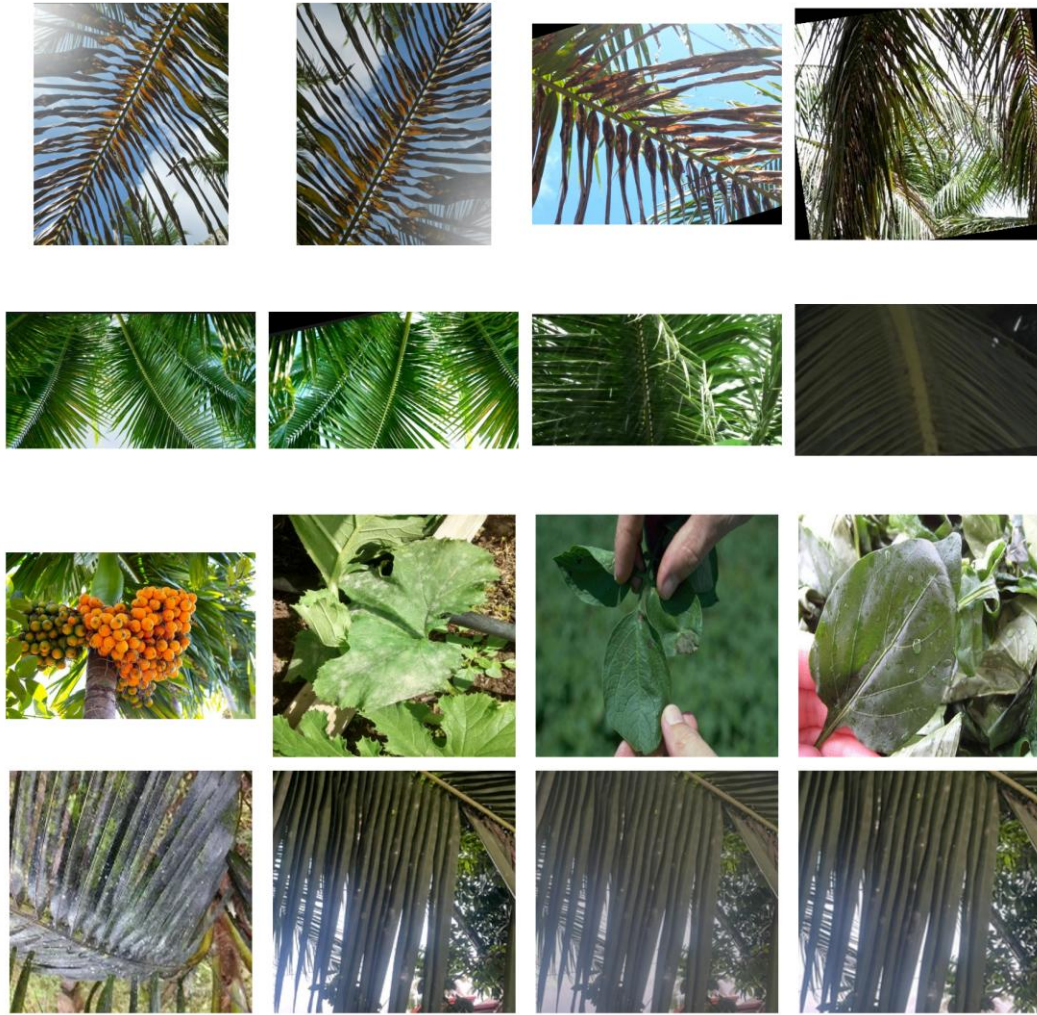
Sample images from each class in the pest detection dataset are presented below, showing representative examples captured by drone under various environmental conditions.

Sample Images from Each Class



Appendix F.1: Sample Images – Mite Model Dataset (coconut_mite | healthy | not_coconut)

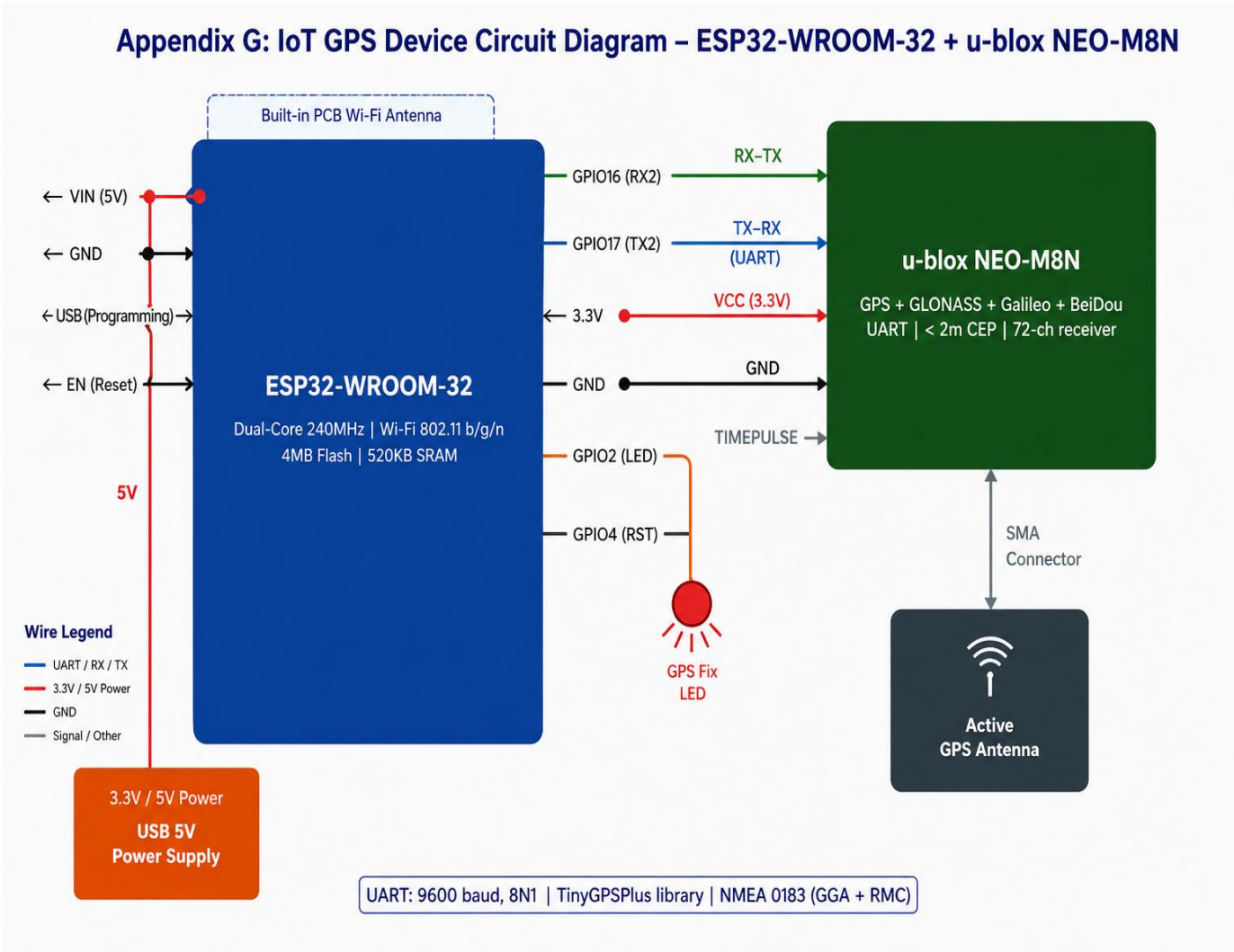
Sample Images from Each Class



Appendix F.2: Sample Images – Unified Model Dataset (caterpillar | healthy | not_coconut | white_fly)

Appendix G – IoT GPS Device Circuit Diagram

The circuit diagram below illustrates the hardware connections between the ESP32-WROOM-32 microcontroller and the u-blox NEO-M8N GNSS module. The two devices communicate via UART at 9600 baud (GPIO16/RX2 ← NEO-TX, GPIO17/TX2 → NEO-RX). The NEO-M8N is powered at 3.3V from the ESP32 voltage regulator. The ESP32 is powered via 5V USB supply.



Appendix H – Flask API v9.0 Endpoint Documentation

The following table documents all endpoints of the CocoSense Flask REST API v9.0 (running on port 5001). All prediction endpoints accept multipart/form-data POST

requests with an 'image' file field (JPEG/PNG, ≤ 10 MB) and return structured JSON responses.

[Table H.1: Flask REST API v9.0 – Complete Endpoint Reference]

Method	Endpoint	Status	Description
GET	/	200 OK	Returns API name, version, and description string.
GET	/health	200 OK	Returns health status of all loaded models. Fields: status, mite_model_loaded, unified_model_loaded, uptime_seconds.
GET	/models	200 OK	Lists all loaded Keras models with class names, input size, and version info.
POST	/predict/mite	200/400	Mite-only prediction using MobileNetV2 Mite Model v10. Applies threshold=0.10, boost=5×. Returns: detected_pest, confidence, description, recommended_action.
POST	/predict/caterpillar	200/400	Caterpillar detection via Unified Model v1 (4-class). Returns caterpillar or healthy/not_coconut if not detected.
POST	/predict/white_fly	200/400	White fly detection via Unified Model v1 (4-class). Returns white_fly or healthy/not_coconut if not detected.
POST	/predict/unified	200/400	Full Unified Model v1 inference across all 4 classes. Returns top class with confidence and per-class probability breakdown.
POST	/predict/all	200/400	Smart combined inference: runs both models + cross-validation logic. Returns final detected_pest, confidence, mite_result, unified_result, cross_validation field.
POST	/predict/disease	200/400	Leaf disease detection (Leaf Rot, Leaf Spot). Uses Disease Detection Model v2 (4-class, 98.69% accuracy).
POST	/predict/leaf-health	200/400	Leaf health classification (healthy/unhealthy) with 9 detailed condition descriptions, urgency level, and treatment steps.
POST	/predict/branch-health	200/400	Branch health classification (healthy/unhealthy) with an unhealthy percentage score. Uses Branch Health Model v1 (99.63% accuracy).
POST	/predict	200/400	Legacy endpoint – redirects to /predict/mite for backwards compatibility.

Common response fields for all /predict/* endpoints:

Table H.2: Common JSON Response Fields

Field	Type	Description
detected_pest	string	Final classification result: coconut_mite caterpillar white_fly healthy not_coconut
confidence	float	Confidence percentage (0–100) for the detected class
severity	string	Severity level: CRITICAL HIGH MODERATE NONE

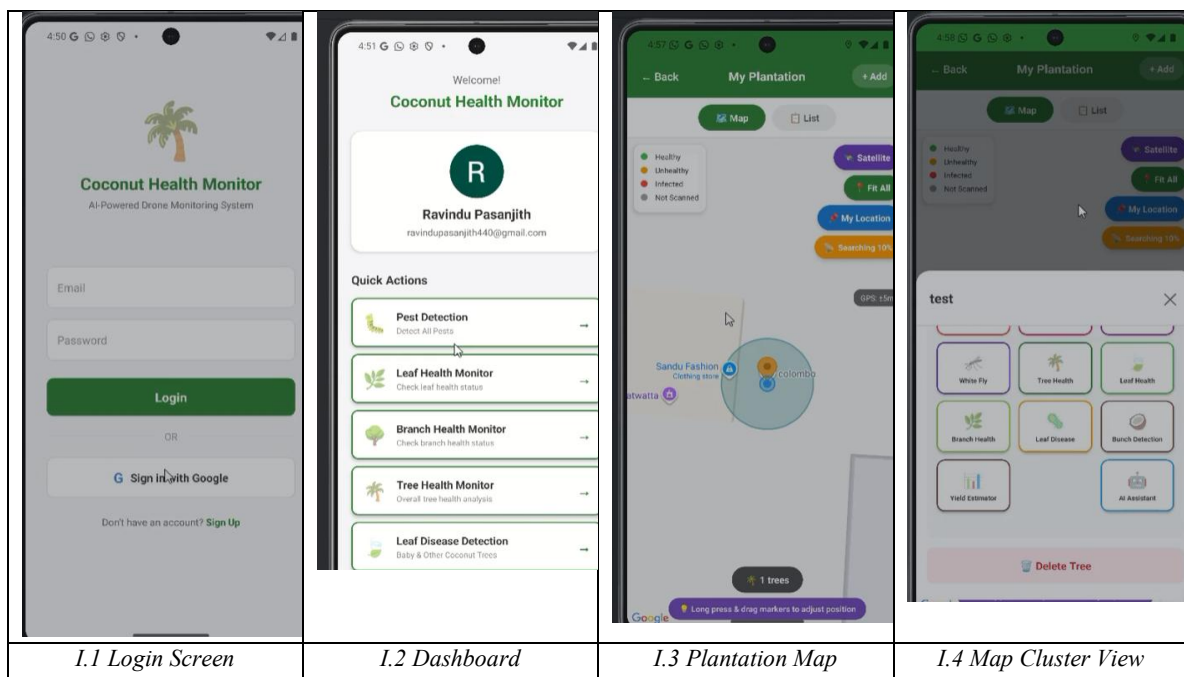
description	string	Human-readable description of the detected pest or condition
recommended_action	string	Specific treatment or monitoring recommendation
mite_result	object	/predict/all only: mite model sub-result {class, confidence, threshold_used, boost_factor}
unified_result	object	/predict/all only: unified model sub-result {class, confidence}
cross_validation	string	/predict/all only: decision pathway taken (mite_accepted mite_fp_suppressed unified_accepted rejected)
response_time_ms	integer	Server-side inference time in milliseconds
error	string	Present only on 400/500 responses. Describes the error (invalid file, unsupported format, etc.)

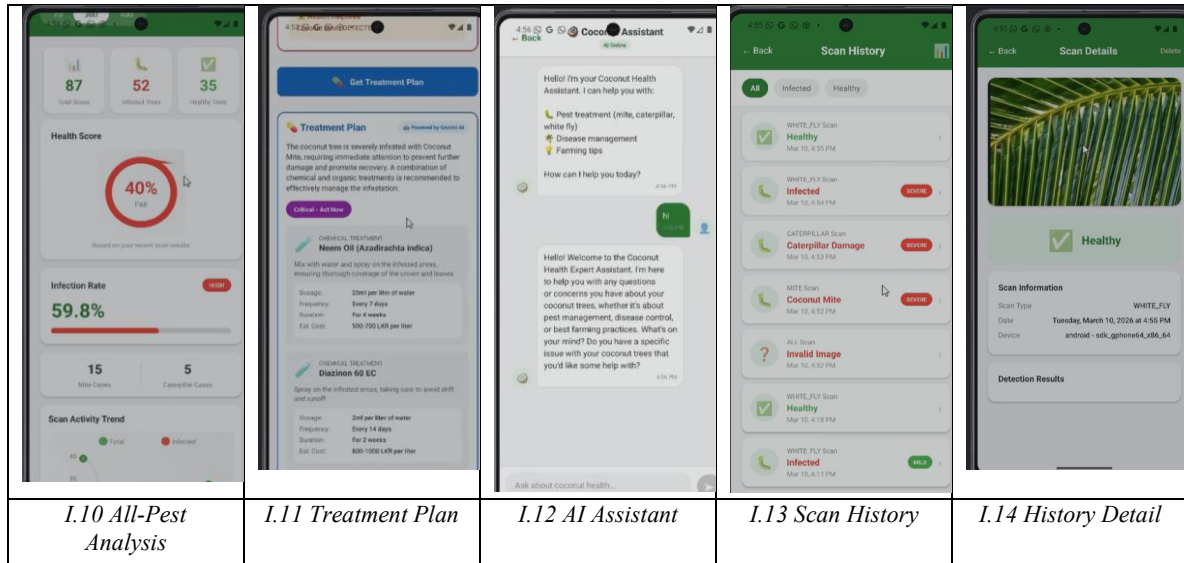
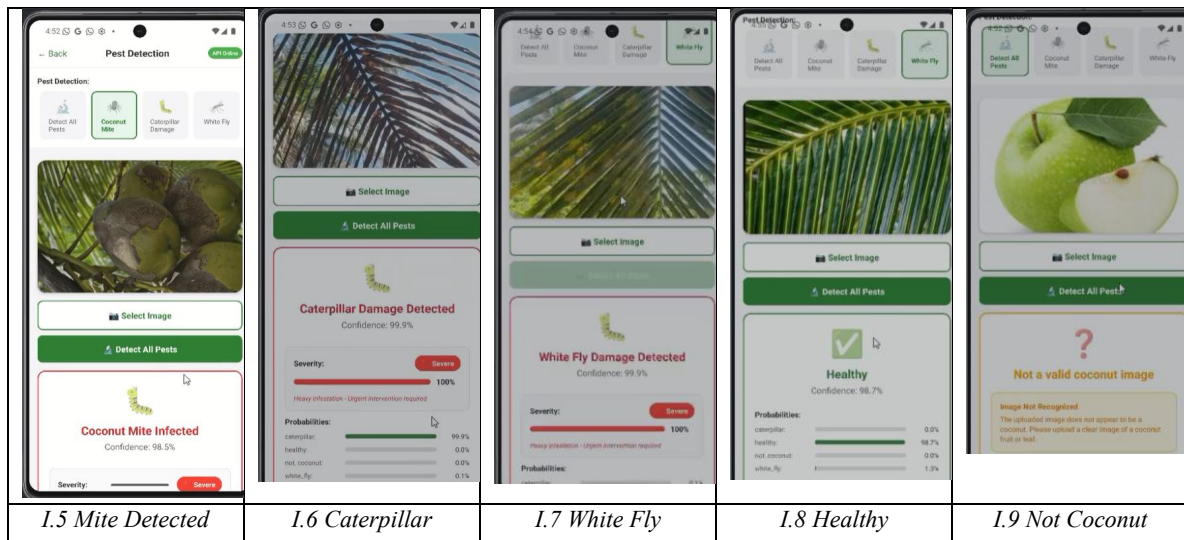
Error response example (HTTP 400):

```
{ "error": "Invalid file type. Only JPEG and PNG are accepted.",
  "status": 400,
  "endpoint": "/predict/all" }
```

Appendix I – Mobile Application Screenshots

Screenshots of the CocoSense React Native mobile application are presented below, covering all key screens of the pest detection workflow.





Appendix J – Plagiarism Report